

ชื่อโครงงาน Agriculture Supply Chain

โดย

67070095	**นางสาวบุษยมาศ**	**สนบ้านเกาะ**
67070112	**นางสาวพัชณกัญญ์**	**คชาวุธากร**
67070131	**นางสาวภารดี**	**พิมลเอกอักษร**
67070147	**นางสาวยานวี**	**ดูเบย์**
67070158	**นางสาววราภรณ์**	**สร้างคำ**
67070178	**นางสาวศุภิสรา**	**หลายชั้น**

เสนอ

ผศ. ดร. พัฒนพงษ์ ฉันทมิตรโอภาส

ผศ. ดร. ลภัส ประดิษฐ์ทัศนีย์

รายงานนี้เป็นส่วนหนึ่งของรหัสวิชา 06026242 Cloud Technology Infrastucture

หลักสูตรวิทยาศาสตร์บัณฑิต สาขาวิทยาการข้อมูลและการวิเคราะห์เชิงธุรกิจ

ภาคเรียนที่ 1 ปีการศึกษา 2569 คณะเทคโนโลยีสารสนเทศ

สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง

คำนำ

ปัจจุบันเทคโนโลยีสารสนเทศมีบทบาทสำคัญต่อการดำเนินงานขององค์กรและธุรกิจเป็นอย่างมาก โดยเฉพาะในยุคที่ข้อมูลมีปริมาณเพิ่มขึ้นอย่างต่อเนื่องและมีความจำเป็นต้องเข้าถึงข้อมูลได้อย่างรวดเร็วและมีประสิทธิภาพ Cloud Technology Infrastructure จึงเข้ามามีบทบาทสำคัญในการสนับสนุนการจัดเก็บประมวลผล และให้บริการทรัพยากรด้านเทคโนโลยีสารสนเทศผ่านเครือข่าย

โครงงาน Agriculture Supply Chain จึงถูกพัฒนาขึ้นเพื่อประยุกต์ใช้แนวคิดด้าน Cloud Technology Infrastructure ในการสนับสนุนการจัดการข้อมูลของห่วงโซ่อุปทานการเกษตร โดยมุ่งเน้นการจัดเก็บและรวบรวมข้อมูลจากกระบวนการต่าง ๆ ให้อยู่ในระบบที่สามารถจัดการและนำไปใช้ประโยชน์ได้อย่างมีประสิทธิภาพ รวมถึงการนำแนวคิด Data Warehouse มาใช้เป็นศูนย์กลางในการจัดเก็บข้อมูลเพื่อรองรับการวิเคราะห์ข้อมูลในภาพรวม เช่น ข้อมูลความต้องการสินค้า ยอดขาย ต้นทุน สินค้าคงคลัง และการสั่งซื้อ

ทั้งนี้ คณะผู้จัดทำหวังว่าโครงงานและรายงานฉบับนี้จะช่วยให้เห็นถึงแนวทางการประยุกต์ใช้ Cloud Technology Infrastructure กับระบบจัดการข้อมูลในภาคการเกษตร และสามารถนำความรู้ที่ได้รับไปต่อยอดในการพัฒนาระบบในอนาคตได้ หากโครงการนี้มีข้อผิดพลาดประการใด ทางคณะผู้จัดทำขออภัยมา ณ ที่นี้

คณะผู้จัดทำ

COPYRIGHT 2026

FACULTY OF INFORMATION TECHNOLOGY

KING MONGKUT'S INSTITUTE OF TECHNOLOGY LADKRABANG

ใบรับรองโครงงาน (PROJECT)

เรื่อง ห่วงโซ่อุปทานสินค้าเกษตร

Agriculture Supply Chain

67070095	**นางสาวบุษยมาศ**	**สนบ้านเกาะ**
67070112	**นางสาวพัชณกัญญ์**	**คชาวุธากร**
67070131	**นางสาวภารดี**	**พิมลเอกอักษร**
67070147	**นางสาวยานวี**	**ดูเบย์**
67070158	**นางสาววราภรณ์**	**สร้างคำ**
67070178	**นางสาวศุภิสรา**	**หลายชั้น**

ขอรับรองว่ารายงานฉบับนี้ ข้าพเจ้าไม่ได้คัดลอกมาจากที่ใด

รายงานฉบับนี้ได้รับการตรวจสอบและอนุมัติให้เป็นส่วนหนึ่งของ

Cloud Technology Infrastructure

หลักสูตรวิทยาศาสตร์บัณฑิต สาขาวิทยาการข้อมูลและการวิเคราะห์เชิงธุรกิจ

ภาคเรียนที่ 1 ปีการศึกษา 2569 คณะเทคโนโลยีสารสนเทศ

คณะผู้จัดทำ

หัวข้อโครงงาน ห่วงโซ่อุปทานสินค้าเกษตร

นักศึกษา นางสาวบุษยมาศ สนบ้านเกาะ

นางสาวพัชณกัญญ์ คชาวุธากร

นางสาวภารดี พิมลเอกอักษร

นางสาวยานวี ดูเบย์

นางสาววราภรณ์ สร้างคำ

นางสาวศุภิสรา หลายชั้น

แขนงวิชา วิศวกรรมข้อมูล

ปีการศึกษา 1/2569

บทคัดย่อ

โครงงานนี้มีวัตถุประสงค์เพื่อแก้ปัญหาการจัดเก็บและการบริหารข้อมูลแบบแยกส่วน (Data Silos) ของสหกรณ์และธุรกิจภาคการเกษตร ซึ่งส่งผลให้ฝ่ายบริหารขาดภาพรวมของห่วงโซ่อุปทานและล่าช้าในการตัดสินใจเชิงกลยุทธ์ คณะผู้จัดทำจึงได้พัฒนาระบบ Data Warehouse แบบรวมศูนย์ (Single Source of Truth) บนโครงสร้างพื้นฐานคลาวด์ Amazon Web Services (AWS) เพื่อเชื่อมโยงข้อมูลตั้งแต่การรับซื้อผลผลิต การจัดการคลังสินค้า การจัดส่ง ไปจนถึงการขายให้อยู่ในระบบเดียวกัน โดยระบบได้พัฒนา Web Application ที่บูรณาการร่วมกับเทคโนโลยี Machine Learning (Predictive Analytics) เพื่อพยากรณ์แนวโน้มทางธุรกิจล่วงหน้า เช่น ปริมาณผลผลิตและราคา พร้อมทั้งติดตั้ง AI Agent (Text-to-SQL) ที่รองรับการประมวลผลภาษาธรรมชาติ (NLP) ช่วยให้ผู้บริหารและนักวิเคราะห์ธุรกิจสามารถพิมพ์คำถามด้วยภาษาพูดเพื่อสืบค้นข้อมูลเชิงลึกและแสดงผลเป็นกราฟฟิกไดนามิก (Dynamic Visualization) ได้ทันทีโดยไม่ต้องเขียนคำสั่ง SQL ผลลัพธ์จากโครงงานนี้ช่วยลดความซ้ำซ้อนในกระบวนการจัดการข้อมูล ขจัดอุปสรรคทางเทคนิคในการเข้าถึงข้อมูลเชิงลึก และช่วยให้ฝ่ายบริหารสามารถวางแผนตัดสินใจได้อย่างแม่นยำและทันท่วงที

กิตติกรรมประกาศ

คณะผู้จัดทำได้เลือกหัวข้อนี้ในการทำงานโครงงาน เนื่องจากความสนใจในงานภาคการเกษตร ซึ่งเล็งเห็นถึงความสำคัญของการนำเทคโนโลยีคลาวด์และการวิเคราะห์ข้อมูลมาช่วยแก้ปัญหา การดำเนินงานโครงงานเรื่อง ห่วงโซ่อุปทานสินค้าเกษตร (Agriculture Supply Chain) สำเร็จลุล่วงไปได้ด้วยดีเป็นผลมาจากการให้คำปรึกษาที่ดีจาก ผศ. ดร. พัฒนพงษ์ ฉันทมิตรโอภาส และ ผศ. ดร. ลภัส ประดิษฐ์ทัศนีย์ อาจารย์ประจำวิชา Cloud Technology Infrastructure ทั้งยังให้ความรู้ในการพัฒนาระบบตลอดการดำเนินงาน

ทั้งนี้ คณะผู้จัดทำขอขอบพระคุณเพื่อนๆ ของคณะผู้จัดทำและคณะเทคโนโลยีสารสนเทศ สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง ที่สนับสนุนการเรียนรู้และอำนวยความสะดวกในการจัดทำโครงงานครั้งนี้

สารบัญ

หน้า

บทคัดย่อ i

กิตติกรรมประกาศ ii

สารบัญ iii

สารบัญรูป vi

สารบัญตาราง viii

บทที่ 1 บทนำ 1

ที่มาและความสำคัญของโครงการ 1

วัตถุประสงค์ของโครงงาน 1

ประโยชน์ที่คาดว่าจะได้รับ 1

ขอบเขตโครงงาน 2

บทที่ 2 ทฤษฎีและเทคโนโลยีที่เกี่ยวข้อง 3

1. ทฤษฎีที่เกี่ยวข้อง 3

2. เทคโนโลยีที่เกี่ยวข้อง 3

บทที่ 3 ขั้นตอนการดำเนินงานของโครงงาน 6

3.1. ความต้องการของระบบ (System Requirements) 6

3.1.1. ความต้องการทางด้านฟังก์ชัน (Functional Requirements) 6

3.1.2. ความต้องการที่ไม่ใช่ฟังก์ชัน (Non-functional Requirements) 6

3.2. การวิเคราะห์และออกแบบระบบ 6

3.2.1. การออกแบบโครงสร้างการใช้งานของ AWS (AWS Architecture Design) 6

3.2.2. การออกแบบคลังข้อมูล (Data Warehouse Design) 8

3.2.3. การออกแบบกระบวนการ ETL (ETL Pipeline Strategy) 8

3.2.4. การออกแบบแผนภาพลำดับการทำงาน (Sequence Diagram) 9

3.3. หลักการทำงานของระบบ 11

3.3.1. การออกแบบหน้าจอผู้ดูแลข้อมูล (Data Administrator Panel) 11

3.3.2. การออกแบบหน้าจอแสดงผลข้อมูลสำหรับผู้บริหาร 11

3.3.3. การออกแบบหน้าต่างปัญญาประดิษฐ์ (AI Agent Interface) 11

3.4. User Interface (Prototype) 12

3.4.1. หน้าเข้าสู่ระบบ (Authentication & Log-in Page) 12

3.4.2. ศูนย์อัปโหลดข้อมูล (Data Upload Center) 13

3.4.3. แผงควบคุมการดำเนินงาน (Visualization Dashboard) 14
3.4.4. ผู้ช่วย AI อัจฉริยะ (Agentic AI Chat) 14
บทที่ 4 ผลการดำเนินงานและระบบต้นแบบ 16
4.1. ส่วนติดต่อผู้ใช้งาน 16
4.1.1. หน้าเข้าสู่ระบบ (Login) 16
4.1.2. Dashboard 17
4.1.3. หน้าคลังสินค้า (นำเข้าข้อมูล CSV) 18
4.1.4. หน้าผู้ช่วย AI (AI Assistant) 18
4.2. การแบ่งสิทธิ์การเข้าถึงตามบทบาทผู้ใช้ (Role-based Access) 19
4.3. ผลการดำเนินงานของการใช้งาน AWS ร่วมกับเว็บแอปพลิเคชัน 21
4.3.1. Amazon VPC และเครือข่าย 21
4.3.2. Security Group 22
4.3.3. Amazon EC2 และ Launch Template 23
4.3.4. Elastic Load Balancing (Application Load Balancer) 23
4.3.5. AWS Auto Scaling 24
4.3.6. Amazon RDS for PostgreSQL 25
4.3.7. Amazon S3 26
4.3.8. Amazon EventBridge และ AWS Step Functions 26
4.3.9. Amazon Lambda และกระบวนการ ETL 27
4.3.10. Amazon DynamoDB 28
4.4. ผลการทดสอบระบบ 28
4.4.1. ผลการนำเข้าข้อมูล (ETL) 28
4.4.2. ผลการทดสอบผู้ช่วย AI 29
4.4.3. ผลการทดสอบความพร้อมใช้งาน 29
4.4.4. ข้อจำกัดของระบบและส่วนที่ไม่เป็นไปตามแผน 30
บทที่ 5. สรุปผลการดำเนินงาน 33
5.1. ที่มาและการทำงานโดยรวมของระบบ 33
5.2. ผลการดำเนินงานเทียบกับวัตถุประสงค์และขอบเขต 33
5.3. ประโยชน์ของระบบ 35
5.4. ปัญหาและอุปสรรคที่เกิดขึ้นในการพัฒนา 35
5.5. ข้อจำกัดของระบบ 35

5.6. ประมาณการค่าใช้จ่าย 36
5.6.1. หลักการและแหล่งข้อมูลที่ใช้ประมาณการ 36
5.6.2. ค่าใช้จ่ายจริงของบัญชีจาก AWS Cost Explorer 36
5.6.3. ค่าใช้จ่ายของโครงงาน AgriChain ถึงปัจจุบัน 37
5.6.4. ประมาณการรายเดือนกรณีเปิดระบบตลอด 24 ชั่วโมง 39
5.6.5. แนวทางลดค่าใช้จ่าย 39
5.7. ข้อเสนอแนะและแนวทางพัฒนาต่อ 40
5.8. สรุป 40
บรรณานุกรม 41
ภาคผนวก 42
ภาคผนวก ก โครงสร้างไฟล์ของโครงการ 42
ภาคผนวก ข โครงสร้างฐานข้อมูลคลังข้อมูล (Star Schema) 43
ภาคผนวก ค ตัวอย่างโค้ด ETL บน AWS Lambda 45
ภาคผนวก ง คู่มือการติดตั้งและการเปิด-ปิดระบบ 47
ภาคผนวก จ พจนานุกรมข้อมูลชุดทดสอบ 47

สารบัญรูป

หน้า

รูปที่ 3.1. AWS Architecture Design..……..6

รูปที่ 3.2. Sequence Diagram...…….....9

รูปที่ 3.3. Use case Diagram...……...……...11

รูปที่ 3.4. Authentication & Log-in……...……..12

รูปที่ 3.5. Data Upload Center…….……...13

รูปที่ 3.6. Visualization Dashboard……..…….....14

รูปที่ 3.7. Agentic AI Chat……...15

รูปที่ 4.1. หน้าเข้าสู่ระบบ (Login)…… 16

รูปที่ 4.2. หน้าเข้าสู่ระบบเมื่อกรอกชื่อผู้ใช้และรหัสผ่าน…….. 17

รูปที่ 4.3. หน้าคลังสินค้า : อัปโหลดไฟล์ CSV และรายการไฟล์ที่นำเข้าแล้ว 8 ไฟล์……...…..17

รูปที่ 4.4. Dashboard……...…….18

รูปที่ 4.5. หน้าผู้ช่วย AI : ตัวอย่างการถามยอดขายสุทธิ ยอดขายรวมตามพืช……...……..19

รูปที่ 4.6. หน้าผู้ช่วย AI เมื่อเปิดดูคำสั่ง SQL ที่ระบบสร้างและใช้ดึงข้อมูล……19

รูปที่ 4.7. มุมมองของ Data Admin (admin01) : เห็นเมนูเฉพาะ "คลังสินค้า" ……...…...20

รูปที่ 4.8. มุมมองของ Data Analyst (analyst01) : เห็นเมนู "แดชบอร์ด" และ "ผู้ช่วย AI" ……...…..21

รูปที่ 4.9. VPC agrichain-vpc และแผนผังทรัพยากร (Resource map)……..21

รูปที่ 4.10. Security Group ของ ALB , เว็บ/AI และฐานข้อมูล (1/3)……. 22

รูปที่ 4.11. Security Group ของ ALB, เว็บ/AI และฐานข้อมูล (2/3)……. 22

รูปที่ 4.12. Security Group ของ ALB, เว็บ/AI และฐานข้อมูล (3/3)…….. 22

รูปที่ 4.13. เครื่อง EC2 ของระบบ……23

รูปที่ 4.14. Application Load Balancer agrichain-alb และ Listener HTTP:80……...23

รูปที่ 4.15. สถานะ Health ของเครื่องใน Target Group agrichain-web-tg……...24

รูปที่ 4.16. Auto Scaling Group agrichain-asg : ประวัติกิจกรรมและนโยบาย Target Tracking (1/2)....24

รูปที่ 4.17. Auto Scaling Group agrichain-asg : ประวัติกิจกรรม และนโยบาย Target Tracking (2/2)...25

รูปที่ 4.18. ฐานข้อมูล RDS agrichain-dwh-db (Publicly accessible = No) (1/2)……. 25

รูปที่ 4.19. ฐานข้อมูล RDS agrichain-dwh-db (Publicly accessible = No) (2/2)……..26

รูปที่ 4.20. ไฟล์ CSV ใน S3 bucket โฟลเดอร์ csv-uploads/……26

รูปที่ 4.21. Graph view ของ Execution ที่สำเร็จ (RunETL ⟶ ETLSucceeded)…….27

รูปที่ 4.22. การตั้งค่า VPC ของ Lambda agrichain-etl (Private Subnet)……..27

รูปที่ 4.23. ตาราง DynamoDB……..28

รูปที่ 5.1. ค่าใช้จ่ายจริงรายวันของบัญชีแยกตามบริการ (ข้อมูล Cost Explorer ก่อนหักเครดิต)…….36

รูปที่ 5.2. งบประมาณของ Learner Lab : ใช้ไปแล้ว $4.4 จากงบ $50…….36

รูปที่ 5.3. องค์ประกอบค่าใช้จ่ายของโครงงาน AgriChain ถึงวันที่ 2 ตุลาคม 2569……..38

รูปที่ข.1. ER Diagram คลังข้อมูล……44

สารบัญตาราง

หน้า

ตารางที่ 4.1. ผู้ใช้ทดสอบและสิทธิ์การเข้าถึงหน้าของแต่ละบทบาท……20

ตารางที่ 4.2. ผลการนำเข้าไฟล์ CSV ด้วย ETL……28

ตารางที่ 4.3. ผลการทดสอบผู้ช่วย AI……..29

ตารางที่ 4.4. ข้อจำกัดและส่วนที่ไม่เป็นไปตามแผน……31

ตารางที่ 5.1. ผลการดำเนินงานเทียบกับวัตถุประสงค์……..34

ตารางที่ 5.2. ผลการดำเนินงานเทียบกับขอบเขตของโครงงาน……34

ตารางที่ 5.3. ค่าใช้จ่ายจริงของบัญชีจาก Cost Explorer แยกตามบริการ (26 ก.ย. – 1 ต.ค. 2569)……...37

ตารางที่ 5.4. ค่าใช้จ่ายโดยประมาณของ stack agrichain ตั้งแต่ติดตั้งจนถึง 2 ต.ค. 2569……...38

ตารางที่ 5.5. ประมาณการรายเดือนเมื่อเปิดระบบตลอด 24 ชั่วโมง...……...……40

ตารางที่ 5.6. แนวทางลดค่าใช้จ่ายและผลโดยประมาณ……40

ตารางที่ จ.1. ไฟล์ข้อมูลทดสอบและคอลัมน์ (* คือคอลัมน์ที่ไม่โหลดเข้าคลังข้อมูล)……...47

บทที่ 1 บทนำ

ที่มาและความสำคัญของโครงการ

ในการดำเนินธุรกิจของสหกรณ์การเกษตรซึ่งทำหน้าที่เป็นศูนย์กลางรับซื้อและจัดจำหน่ายผลิตผลทางการเกษตร ข้อมูลการปฏิบัติงานในแต่ละขั้นตอน เช่น การรับซื้อจากเกษตรกร การจัดการคลังสินค้า การจัดส่ง และการขาย มักจะถูกจัดเก็บแยกส่วนกันในแต่ละแผนก การแยกส่วนของระบบเหล่านี้ส่งผลให้ฝ่ายบริหารไม่สามารถมองเห็นภาพรวมของธุรกิจได้อย่างครบถ้วน ทำให้การวิเคราะห์ประสิทธิภาพการดำเนินงานและการตัดสินใจเชิงกลยุทธ์เป็นไปอย่างล่าช้าและขาดความแม่นยำ

จากปัญหาดังกล่าว ผู้จัดทำจึงมีแนวคิดในการพัฒนาโครงการแพลตฟอร์ม Data Warehouse แบบรวมศูนย์ บนระบบ Cloud (AWS) เพื่อทำหน้าที่เป็นแหล่งข้อมูลที่ถูกต้องเพียงแหล่งเดียวขององค์กร โดยระบบจะทำการดึงข้อมูล ทำความสะอาด และแปลงรูปแบบข้อมูล จากไฟล์บันทึกการปฏิบัติงานของแต่ละแผนกมารวมไว้ในฐานข้อมูลเชิงวิเคราะห์ โครงการนี้ไม่เพียงแต่ช่วยลดอุปสรรคในการเชื่อมโยงข้อมูลข้ามแผนก สามารถนำไปต่อยอดแสดงผลผ่าน Business Dashboard ช่วยให้ผู้ใช้งานระดับบริหารสามารถติดตามแนวโน้มและวิเคราะห์ข้อมูลทางธุรกิจได้อย่างมีประสิทธิภาพ

วัตถุประสงค์ของโครงงาน

1. เพื่อให้องค์กรมีศูนย์กลางข้อมูล สำหรับดูภาพรวมและวิเคราะห์ประสิทธิภาพการดำเนินงานได้อย่างบูรณาการ
2. เพื่อพัฒนาระบบคลังข้อมูลบนเทคโนโลยี Cloud ที่สามารถนำเข้า ทำความสะอาด และประมวลผลข้อมูลได้แบบอัตโนมัติ
3. เพื่อประยุกต์ใช้เทคโนโลยีปัญญาประดิษฐ์ (AI Agent) และการเรียนรู้ของเครื่อง (Machine Learning) ในการพยากรณ์แนวโน้มทางธุรกิจและโต้ตอบกับข้อมูลผ่านภาษาธรรมชาติ
4. เพื่อสร้างเว็บแอปพลิเคชัน (Web Application) ที่เปิดให้ผู้ใช้งานสามารถติดตามตัวชี้วัด (KPIs) และสื่อสารเพื่อดึงข้อมูลเชิงลึก (Insights) ไปประกอบการตัดสินใจเชิงกลยุทธ์ได้อย่างรวดเร็ว

ประโยชน์ที่คาดว่าจะได้รับ

1. ผู้ใช้งานระดับบริหารและนักวิเคราะห์มีระบบอัจฉริยะที่ช่วยคาดการณ์แนวโน้มล่วงหน้า (Predictive Insights) ทำให้สามารถวางแผนกลยุทธ์ได้แม่นยำกว่าการดูข้อมูลในอดีตเพียงอย่างเดียว
2. ลดภาระการทำงานซ้ำซ้อนของผู้ดูแลข้อมูล ด้วยท่อส่งข้อมูลอัตโนมัติ (Automated ETL) ที่อัปเดตข้อมูลเข้าสู่คลังข้อมูลทันทีเมื่อมีไฟล์ใหม่เข้าสู่ระบบ

3. การสืบค้นข้อมูลมีความสะดวกรวดเร็วขึ้นผ่าน AI Agent ซึ่งช่วยลดอุปสรรคทางเทคนิคสำหรับผู้บริหารที่ไม่มีความเชี่ยวชาญในการเขียนคำสั่งฐานข้อมูล (SQL) หรือการใช้เครื่องมือที่ซับซ้อน
4. ได้ระบบต้นแบบสถาปัตยกรรมข้อมูลสมัยใหม่ บน Cloud ที่มีการใช้ทรัพยากรอย่างคุ้มค่า ยืดหยุ่น และพร้อมรองรับปริมาณข้อมูลที่เพิ่มขึ้นในอนาคต

ขอบเขตโครงงาน

1. **การจัดการข้อมูลอัตโนมัติ (Automated Data Integration & ETL) :** พัฒนากระบวนการดึงข้อมูล ทำความสะอาด และแปลงข้อมูลแบบอัตโนมัติ (Event-driven) ทันทีที่มีการอัปโหลดไฟล์ปฏิบัติการเข้าสู่พื้นที่จัดเก็บ (เช่น Amazon S3)
2. **คลังข้อมูล (Data Warehouse) :** ออกแบบฐานข้อมูลเชิงวิเคราะห์รูปแบบ Star Schema ประกอบด้วย Fact Tables (เช่น การรับซื้อ, การขาย, การจัดส่ง) และ Dimension Tables โดยใช้ **Amazon Redshift** เพื่อรองรับการสืบค้นข้อมูลระดับองค์กร
3. **การวิเคราะห์เชิงทำนาย (Predictive Analytics) :** พัฒนาโมเดล Machine Learning เพื่อคาดการณ์และพยากรณ์แนวโน้มทางธุรกิจในอนาคต (เช่น แนวโน้มราคาหรือปริมาณผลผลิต)
4. **การประมวลผลปัญญาประดิษฐ์ (AI Agent) :** พัฒนาระบบโต้ตอบด้วยภาษาธรรมชาติ (Text-to-SQL) โดยประยุกต์ใช้ Large Language Model (LLM) เพื่อแปลงคำถามของผู้ใช้งานเป็นคำสั่งสืบค้นข้อมูล
5. **เว็บแอปพลิเคชัน (Web Application) :** พัฒนาแพลตฟอร์มแบบ Custom Web Application ที่ประกอบด้วยระบบยืนยันตัวตน (Authentication) หน้าแสดงผลกราฟฟิกแดชบอร์ดหลัก (Default Dashboard) และหน้าต่างแชทสำหรับโต้ตอบกับ AI
6. **สถาปัตยกรรมระบบ Cloud (Cloud Architecture) :** ติดตั้งและใช้งานโครงสร้างพื้นฐานบน Amazon Web Services (AWS) โดยใช้บริการหลัก ได้แก่ S3, EC2, Lambda และ Redshift พร้อมตั้งค่าสถาปัตยกรรมให้รองรับ Auto Scaling เพื่อความเสถียรของระบบ

บทที่ 2 ทฤษฎีและเทคโนโลยีที่เกี่ยวข้อง

1. ทฤษฎีที่เกี่ยวข้อง

1.1. คลังข้อมูล (Data Warehouse) และ Single Source of Truth (SSOT)

Data Warehouse คือ ระบบจัดการข้อมูลที่ถูกออกแบบมาเพื่อรองรับการประมวลผลเชิงวิเคราะห์ มากกว่าการประมวลผลธุรกรรมเชิงปฏิบัติการ ในโครงงานนี้ Data Warehouse ทำหน้าที่เป็นแหล่งข้อมูลที่ถูกต้องเพียงแหล่งเดียว ที่รวบรวมข้อมูลจากระบบปฏิบัติการที่แยกส่วนกัน เช่น ระบบรับซื้อ, ระบบสินค้าคงคลัง, ระบบจัดส่ง, และระบบการขาย เพื่อให้ฝ่ายบริหารสามารถวิเคราะห์ภาพรวมทางธุรกิจได้

1.2. กระบวนการ ETL (Extract, Transform, Load)

- **Extract :** การดึงข้อมูลดิบในรูปแบบไฟล์ CSV จากแหล่งข้อมูลระบบปฏิบัติการต่าง ๆ
- **Transform :** การทำความสะอาดข้อมูลตรวจสอบความถูกต้องกำจัดข้อมูลซ้ำซ้อน และจัดการข้อมูลที่สูญหายให้อยู่ในรูปแบบที่พร้อมใช้งาน
- **Load :** การนำข้อมูลที่ผ่านการแปลงรูปแล้วโหลดเข้าสู่ฐานข้อมูลปลายทาง (Data Warehouse) ในโครงงานนี้ กระบวนการ ETL จะทำงานในรูปแบบ Daily Batch Processing เพื่อนำเข้าข้อมูลรายวัน

1.3. การออกแบบแบบจำลองคลังข้อมูล (Star Schema Design)

Star Schema เป็นหลักการออกแบบฐานข้อมูลเชิงวิเคราะห์ เน้นการสืบค้นข้อมูลเชิงประวัติ (Historical Analysis) ประกอบด้วย 2 ส่วน:

- **Fact Tables :** ตารางหลักที่ใช้เก็บข้อมูลเชิงปริมาณหรือเหตุการณ์ทางธุรกิจที่เกิดขึ้น
- **Dimension Tables :** ตารางที่เก็บข้อมูลบริบทหรือรายละเอียดใช้ในการกรองและจัดกลุ่มข้อมูล

1.4. ระบบอัจฉริยะทางธุรกิจ (Business Intelligence - BI)

การนำข้อมูลมาแสดงผลในรูปแบบ Dashboard และรายงานเชิงวิเคราะห์ เพื่อช่วยให้ผู้ใช้งานกลุ่ม Executive และ Business Analyst สามารถติดตามตัวชี้วัด วิเคราะห์แนวโน้ม และสนับสนุนการตัดสินใจเชิงกลยุทธ์ขององค์กร

2. เทคโนโลยีที่เกี่ยวข้อง

2.1. สถาปัตยกรรมคลาวด์เทคโนโลยีจาก AWS (Amazon Web Services)

- **Amazon EC2 (Elastic Compute Cloud) :** บริการเซิร์ฟเวอร์เสมือนบนคลาวด์ ใช้สำหรับเป็นหน่วยประมวลผลหลักในการรันสคริปต์ Python สำหรับกระบวนการ ETL และรันระบบ Backend ด้วย Node.js และ Express.js

- **Amazon DynamoDB :** บริการฐานข้อมูล NoSQL แบบ Serverless ที่มีความรวดเร็วสูงและยืดหยุ่น โครงงานนี้เลือกใช้สำหรับจัดเก็บข้อมูลผู้ใช้งานและตรวจสอบสิทธิ์การเข้าสู่ระบบ (Authentication & Session Management) เพื่อแยกภาระงานออกจากคลังข้อมูลหลัก
- **Amazon Redshift :** บริการคลังข้อมูล (Data Warehouse) ระดับองค์กรที่มีการประมวลผลข้อมูลเชิงวิเคราะห์ (OLAP) บนคลาวด์ รองรับการจัดเก็บข้อมูลจำนวนมหาศาล และประมวลผลคำสั่ง SQL เชิงซ้อนด้วยความเร็วสูง โครงงานนี้ใช้เป็นศูนย์กลางข้อมูลสำหรับแบบจำลอง Star Schema
- **Amazon S3 (Simple Storage Service) :** บริการพื้นที่จัดเก็บข้อมูล Object Storage ใช้เป็นพื้นที่อัปโหลดและจัดเก็บชุดข้อมูลปฏิบัติการต้นทางในรูปแบบ CSV ก่อนเข้าสู่กระบวนการ ETL
- **AWS Step Functions และ AWS Lambda :** สถาปัตยกรรม Serverless ที่ทำงานร่วมกัน โดย AWS Step Functions ทำหน้าที่เป็นผู้กำกับลำดับงาน (Orchestrator) สั่งการให้ AWS Lambda ทำงานย่อยในการทำความสะอาดและแปลงข้อมูล ทันทีที่มีชุดข้อมูลอัปโหลดเข้า Amazon S3
- **Application Load Balancer (ALB) :** บริการกระจายปริมาณการเข้าใช้งานไปยัง EC2 Instance ต่าง ๆ อย่างเหมาะสม เพื่อไม่ให้เซิร์ฟเวอร์ตัวใดตัวหนึ่งรับภาระหนักเกินไป
- **Auto Scaling :** บริการที่ช่วยปรับขนาดและจำนวนของ EC2 Instance โดยอัตโนมัติ เพื่อรองรับปริมาณงานหรือการประมวลผลข้อมูลที่สูงขึ้นในช่วงเวลาที่กำหนด
- **Multi-AZ (Multi Availability Zones) :** การออกแบบสถาปัตยกรรมให้ครอบคลุมหลายศูนย์ข้อมูล (Data Center) เพื่อความพร้อมใช้งานสูง และป้องกันข้อมูลสูญหายกรณีเกิดเหตุขัดข้อง

2.2. เทคโนโลยีและเครื่องมืออื่น ๆ ที่ใช้

- **Python :** ภาษาโปรแกรมหลักที่ใช้ในการเขียนสคริปต์สร้าง ETL Pipeline เนื่องจากมีไลบรารีที่สนับสนุนการจัดการและแปลงรูปข้อมูลอย่างมีประสิทธิภาพ
- **Node.js และ Express.js :** เทคโนโลยีฝั่ง Backend ที่ใช้ JavaScript ในการทำงาน เพื่อพัฒนา API และระบบจัดการเบื้องหลังของแพลตฟอร์ม
- **Docker :** เครื่องมือสร้าง Containerization ช่วยในการแพ็กเกจแอปพลิเคชันและสภาพแวดล้อมการทำงานให้อยู่ในรูปแบบ Container เพื่อให้สามารถนำไปรันบน Amazon EC2 ได้อย่างสะดวกและลดปัญหาความเข้ากันได้ของระบบ
- **การวิเคราะห์เชิงทำนาย (Predictive Analytics) :** เทคนิคทางสถิติและ Machine Learning เพื่อวิเคราะห์ข้อมูลในอดีตและปัจจุบันสำหรับค้นหาความสัมพันธ์และทำนายเหตุการณ์ที่อาจเกิดขึ้นในอนาคต นำมาใช้คาดการณ์แนวโน้มทางธุรกิจเพื่อสนับสนุนการตัดสินใจเชิงกลยุทธ์

- **ปัญญาประดิษฐ์และการประมวลผลภาษาธรรมชาติ (AI & NLP) :** เทคโนโลยีด้านการประมวลผลภาษาธรรมชาติ นำมาประยุกต์ใช้เป็น AI Agent ทำหน้าที่แปลความหมายจากข้อความภาษาคน ให้เป็นคำสั่งสืบค้นฐานข้อมูล ช่วยให้ผู้ใช้งานสืบค้นข้อมูลเชิงลึกได้โดยตรง
- **สถาปัตยกรรมการประมวลผลแบบขับเคลื่อนด้วยเหตุการณ์ (Event-Driven Architecture) :** รูปแบบการทำงานที่ตอบสนองต่อเหตุการณ์ (Event) แบบเรียลไทม์ ในกระบวนการ ETL ระบบจะถูกกระตุ้นให้ทำงานทันทีเมื่อมีการอัปโหลดไฟล์เข้าสู่พื้นที่จัดเก็บ ช่วยให้ข้อมูลมีความทันสมัยและลดการใช้ทรัพยากรประมวลผล

บทที่ 3 ขั้นตอนการดำเนินงานของโครงงาน

3.1. ความต้องการของระบบ (System Requirements)

จากการวิเคราะห์กระบวนการดำเนินธุรกิจของสหกรณ์การเกษตร ได้กำหนดความต้องการของระบบออกเป็น 2 ส่วนหลัก เพื่อให้ครอบคลุมทั้งด้านการใช้งานและด้านประสิทธิภาพ ดังนี้

3.1.1. ความต้องการทางด้านฟังก์ชัน (Functional Requirements)

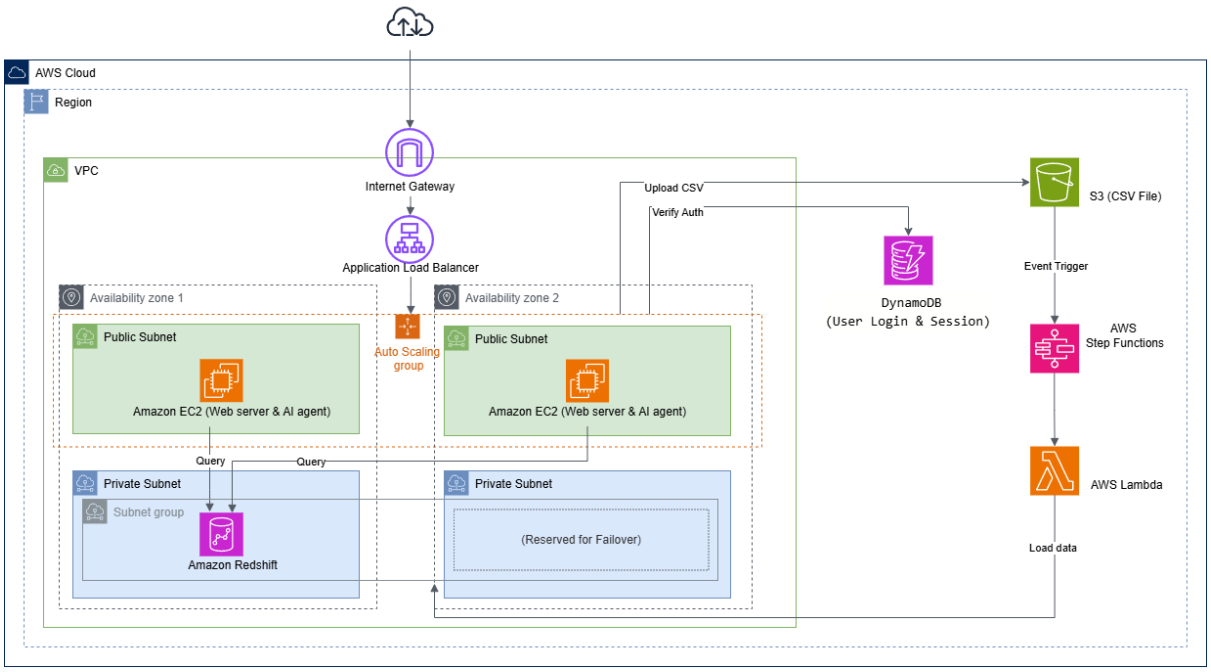
- ระบบมีฟังก์ชันยืนยันตัวตน (Authentication) เพื่อรักษาความปลอดภัยของข้อมูล
- ระบบรองรับการนำเข้าไฟล์ข้อมูลปฏิบัติการ พร้อมกระตุ้นให้ทำงานอัตโนมัติ เมื่อมีข้อมูลใหม่
- ระบบกระบวนการ ETL อัตโนมัติ เพื่อตรวจสอบ ทำความสะอาด แปลงข้อมูลเข้าสู่คลังข้อมูล
- ระบบจัดเก็บข้อมูลในคลังข้อมูลระดับองค์กร (Amazon Redshift) ด้วย Star Schema
- ระบบมีโมเดลการเรียนรู้ของเครื่อง (Machine Learning) เพื่อวิเคราะห์เชิงทำนาย
- ระบบมี AI Agent รองรับการประมวลผลภาษาธรรมชาติ (Text-to-SQL) เพื่อให้ผู้บริหารพิมพ์โต้ตอบสืบค้นข้อมูลผ่าน Web Application ได้โดยตรง

3.1.2. ความต้องการที่ไม่ใช่ฟังก์ชัน (Non-functional Requirements)

- ระบบต้องทำงานบนโครงสร้างพื้นฐานของ AWS (Amazon Web Services) โดยใช้สถาปัตยกรรมแบบ Auto Scaling และรองรับความพร้อมใช้งาน

3.2. การวิเคราะห์และออกแบบระบบ

3.2.1. การออกแบบโครงสร้างการใช้งานของ AWS (AWS Architecture Design)



รูปที่ 3.1. AWS Architecture Design

การออกแบบโครงสร้างพื้นฐานสำหรับระบบ บน Amazon Web Services (AWS) ถูกออกแบบโดยยึดหลัก ความพร้อมใช้งานสูง, ความปลอดภัยของข้อมูล, และ ความยืดหยุ่นในการขยายระบบ โดยแบ่งสถาปัตยกรรมออกเป็น 4 ส่วนหลัก ดังนี้

1. การออกแบบระบบเครือข่ายและความปลอดภัย (Network & Security Design)

เพื่อให้ระบบมีความปลอดภัยและแยกสัดส่วนการเข้าถึงข้อมูลอย่างชัดเจน ระบบจึงถูกสร้างขึ้นภายในเครือข่ายเสมือนส่วนตัว หรือ Amazon VPC (Virtual Private Cloud) โดยแบ่ง Subnets ออกเป็น 2 ประเภท ครอบคลุมพื้นที่ 2 Availability Zones เพื่อป้องกันเหตุขัดข้องจากศูนย์ข้อมูล:

- **Public Subnet :** เป็นพื้นที่ที่เชื่อมต่อกับอินเทอร์เน็ตผ่าน Internet Gateway ใช้สำหรับวางเครื่องเซิร์ฟเวอร์ประมวลผลเว็บแอปพลิเคชันและปัญญาประดิษฐ์ (EC2) และตัวกระจายภาระงาน (Application Load Balancer) เพื่อให้สามารถรับคำสั่งจากผู้ใช้งานและผู้บริหารได้
- **Private Subnet :** ไม่สามารถเข้าถึงได้โดยตรงจากอินเทอร์เน็ต ใช้สำหรับวางคลังข้อมูลเชิงวิเคราะห์ (Amazon Redshift) เพื่อป้องกันการโจมตีจากภายนอก โดยคลังข้อมูลจะรับการเชื่อมต่อจากเครื่อง EC2 หรือกระบวนการของ AWS Lambda ที่อยู่ภายใน VPC เดียวกันเท่านั้น
- **Security Groups :** มีการตั้งค่า Security Group ของคลังข้อมูล (Amazon Redshift) ให้เปิดรับพอร์ต 5439 เฉพาะจากการเชื่อมต่อของ Security Group ของเครื่อง EC2 และ AWS Lambda ที่ได้รับอนุญาตเท่านั้น
- **Authentication & Gateway :** ผู้ใช้งานเชื่อมต่อผ่าน Internet Gateway และ Application Load Balancer โดยเซิร์ฟเวอร์ EC2 จะทำการตรวจสอบสิทธิ์การเข้าใช้งานกับ Amazon DynamoDB ก่อนอนุญาตให้เข้าถึงข้อมูล

2. การออกแบบระบบประมวลผลแบบผสมผสาน (Compute & Serverless Design) สถาปัตยกรรมถูกออกแบบให้แยกส่วนการทำงาน ระหว่างระบบหน้าบ้านและระบบจัดการข้อมูลหลังบ้าน เพื่อประสิทธิภาพสูงสุดและประหยัดค่าใช้จ่าย:

- **Amazon EC2 และ Auto Scaling :** เป็นหน่วยประมวลผลหลักสำหรับระบบ Backend (Node.js/Python) เพื่อทำหน้าที่แสดงหน้า Dashboard, ประมวลผลโมเดล Machine Learning และจัดการคำสั่งของ AI Agent โดยทำงานร่วมกับ Application Load Balancer (ALB) และ Auto Scaling Group เพื่อเพิ่มจำนวนเครื่องอัตโนมัติหากมีปริมาณผู้ใช้งานจำนวนมาก
- **AWS Step Functions และ AWS Lambda :** ทำหน้าที่จัดการกระบวนการ ETL แทน EC2 โดยทำงานในรูปแบบไร้เซิร์ฟเวอร์ (Serverless) เพื่อสกัด ทำความสะอาด และแปลงรูปข้อมูลอัตโนมัติ ช่วยลดภาระการทำงานของเซิร์ฟเวอร์หลักได้

3. จัดเก็บข้อมูลและนำเข้าอัตโนมัติ (Data Ingestion Design)

ก่อนที่ข้อมูลปฏิบัติการจากแต่ละแผนกจะถูกนำเข้าสู่คลังข้อมูล จะต้องผ่านการจัดเก็บและกระตุ้นการทำงาน

- **Amazon S3 :** ทำหน้าที่เป็น Landing Zone สำหรับจัดเก็บไฟล์ข้อมูลดิบในรูปแบบ CSV โดย Amazon S3 จะอยู่นอก VPC
- **Event-driven Automation :** ทันทีที่มีไฟล์ถูกอัปโหลดเข้าสู่ Amazon S3 ระบบจะส่งสัญญาณ (Event Trigger) ไปกระตุ้นให้ AWS Step Functions และ AWS Lambda เข้ามาดึงข้อมูลไปทำความสะอาด และแปลงรูป อย่างรวดเร็วและปลอดภัยโดยไม่ต้องรอผู้ดูแลระบบสั่งการ

4. การออกแบบคลังข้อมูลเชิงวิเคราะห์ (Data Warehouse Design)

การจัดเก็บข้อมูลที่พร้อมสำหรับการวิเคราะห์และรองรับระบบปัญญาประดิษฐ์:

- **Amazon Redshift :** ทำหน้าที่เป็นคลังข้อมูลหลักระดับ OLAP (Enterprise Data Warehouse) จัดเก็บข้อมูลในโครงสร้าง Star Schema รองรับการคิวรีข้อมูลขนาดใหญ่ด้วยความเร็วสูง
- **High Availability & Multi-AZ :** ระบบคลังข้อมูลถูกตั้งค่าให้มีความพร้อมใช้งานสูง (High Availability) และกระจายการทำงานข้าม Availability Zones เพื่อรองรับความเสียหาย หากศูนย์ข้อมูลหลักขัดข้อง ระบบสามารถทำงานต่อได้โดยอัตโนมัติ ทำให้ Dashboard และหน้าต่างโต้ตอบของ AI Agent สามารถดึงข้อมูลไปแสดงผลได้อย่างต่อเนื่อง

3.2.2. การออกแบบคลังข้อมูล (Data Warehouse Design)

คลังข้อมูลถูกออกแบบแบบ Star Schema เพื่อรองรับการค้นข้อมูลเชิงวิเคราะห์โดยแยกตารางเป็น

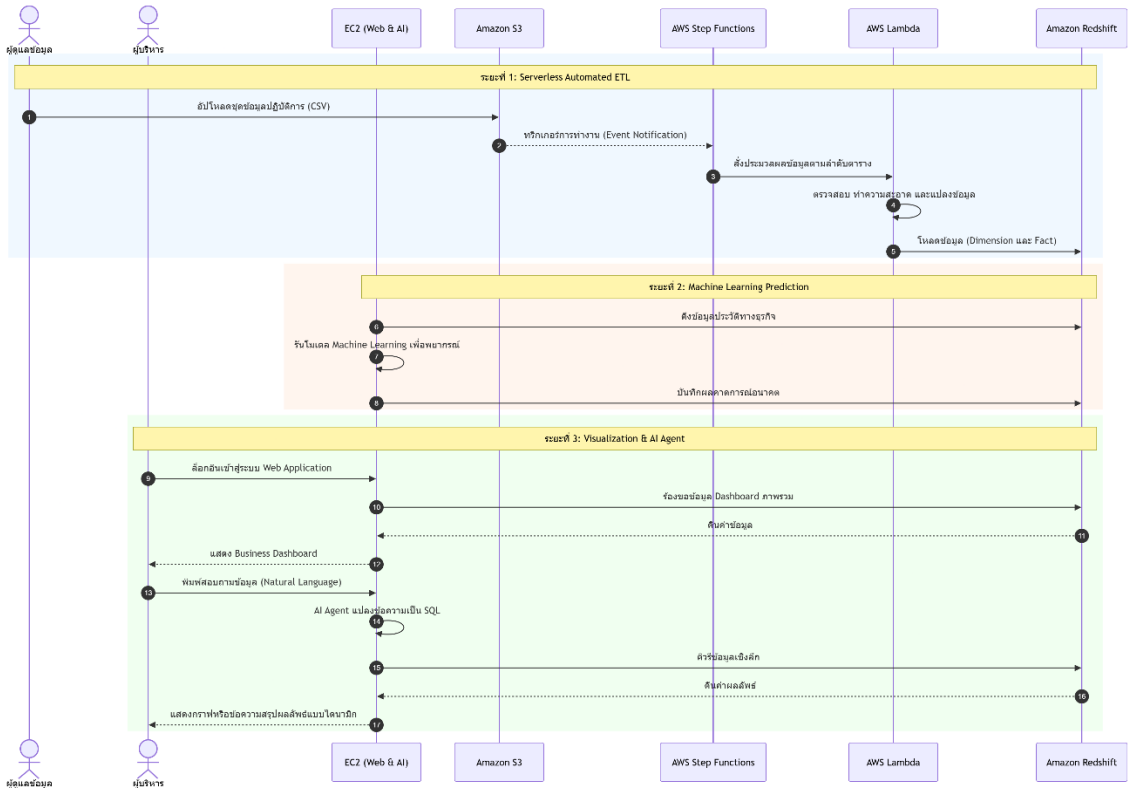
- **Fact Tables (ตารางข้อเท็จจริง) :** จำนวน 3 ตารางหลัก ได้แก่ Harvest Fact (ข้อมูลการรับซื้อ), Sales Fact (ข้อมูลยอดขาย), และ Shipment Fact (ข้อมูลการจัดส่ง)
- **Dimension Tables (ตารางมิติ) :** ตารางที่ใช้เป็นเงื่อนไขในการวิเคราะห์ ได้แก่ Date (วันที่), Farmer (เกษตรกร), Crop (ผลผลิต), Warehouse (คลังสินค้า), Province (จังหวัด), และ Customer (ลูกค้า)

3.2.3. การออกแบบกระบวนการ ETL (ETL Pipeline Strategy)

กระบวนการไหลของข้อมูล (Data Flow) ถูกกำหนดไว้ดังนี้ :

1. **Export & Upload :** แต่ละระบบปฏิบัติการนำออกข้อมูลเป็นไฟล์ CSV และผู้ดูแลข้อมูล (Data Administrator) ทำการอัปโหลดไฟล์เข้าสู่ระบบ
2. **Processing :** สคริปต์ Python เริ่มกระบวนการ ETL ดึงไฟล์จาก AWS S3 มาทำความสะอาด กำจัดค่าที่ผิดปกติหรือซ้ำซ้อน และบันทึกข้อผิดพลาด
3. **Loading :** นำข้อมูลที่สมบูรณ์โหลดเข้าสู่ระบบฐานข้อมูล PostgreSQL บน Amazon RDS

3.2.4. การออกแบบแผนภาพลำดับการทำงาน (Sequence Diagram)



รูปที่ 3.2. Sequence Diagram

แบ่งขั้นตอนการทำงานออกเป็น 3 ระยะหลัก ดังนี้ :

- **ระยะที่ 1 : การนำเข้าและประมวลผลข้อมูลอัตโนมัติ (Serverless Automated ETL)**

  ระยะเตรียมการ : การเข้าสู่ระบบและการตรวจสอบสิทธิ์ (Authentication) ผู้ใช้งาน/ผู้ดูแลระบบกรอกข้อมูลเข้าสู่ระบบผ่าน Web Application เซิร์ฟเวอร์ EC2 ส่งข้อมูลไปตรวจสอบความถูกต้องที่ฐานข้อมูล Amazon DynamoDB เมื่อรหัสผ่านถูกต้อง ระบบจะอนุญาตให้เข้าสู่หน้า Dashboard หรือหน้าจัดการข้อมูล

  ขั้นตอนที่ 1-2 (Upload CSV datasets) : ผู้ดูแลข้อมูล (Data Admin) ทำการอัปโหลดไฟล์ชุดข้อมูลปฏิบัติการในรูปแบบ CSV เข้าสู่พื้นที่จัดเก็บข้อมูล Amazon S3 เมื่อระบบจัดเก็บไฟล์เสร็จสิ้น S3 จะส่งสัญญาณกระตุ้นการทำงาน (Event Notification) ไปยังระบบประมวลผลทันที

  ขั้นตอนที่ 3-4 (Trigger & Extract) : AWS Step Functions รับสัญญาณและทำหน้าที่เป็นผู้กำกับลำดับงาน (Orchestrator) สั่งการให้ AWS Lambda ทำการดึงข้อมูลดิบจาก S3 มาประมวลผลตามลำดับเงื่อนไขที่กำหนดไว้

ขั้นตอนที่ 5-6 (Clean, Validate, Transform) : ระบบ AWS Lambda ทำการทำความสะอาดข้อมูล ตรวจสอบความถูกต้องโครงสร้าง ลบข้อมูลที่ซ้ำซ้อน และแปลงให้อยู่ในรูปแบบโครงสร้างแบบจำลอง Star Schema

ขั้นตอนที่ 7-8 (Load to Redshift) : AWS Lambda โหลดข้อมูลที่ผ่านการแปลงรูปแล้ว (ตาราง Dimension และ Fact) เข้าสู่ระบบคลังข้อมูล Amazon Redshift เมื่อกระบวนการเสร็จสิ้น ระบบจะมีการบันทึกสถานะการทำงาน (Logs) ไว้เบื้องหลัง

- **ระยะที่ 2 : การวิเคราะห์เชิงทำนาย (Machine Learning Prediction)**

  ขั้นตอนที่ 9 (Query Historical Data) : เซิร์ฟเวอร์ EC2 (Web & AI Backend) ส่งคำสั่งสืบค้นข้อมูลประวัติการดำเนินงานจากคลังข้อมูล Amazon Redshift เพื่อนำมาเป็นข้อมูลตั้งต้น

  ขั้นตอนที่ 10 (Run ML Model) : เซิร์ฟเวอร์รันโมเดลการเรียนรู้ของเครื่อง (Machine Learning) ที่เตรียมไว้ เพื่อประมวลผลและคาดการณ์แนวโน้มทางธุรกิจล่วงหน้า

  ขั้นตอนที่ 11 (Save Predictions) : เซิร์ฟเวอร์ EC2 นำผลลัพธ์การคาดการณ์ที่ได้จากการคำนวณบันทึกกลับลงสู่ตารางจัดเก็บใน Amazon Redshift เพื่อรอการนำไปแสดงผล

- **ระยะที่ 3 : การแสดงผลและโต้ตอบด้วย AI (Visualization & AI Agent)**

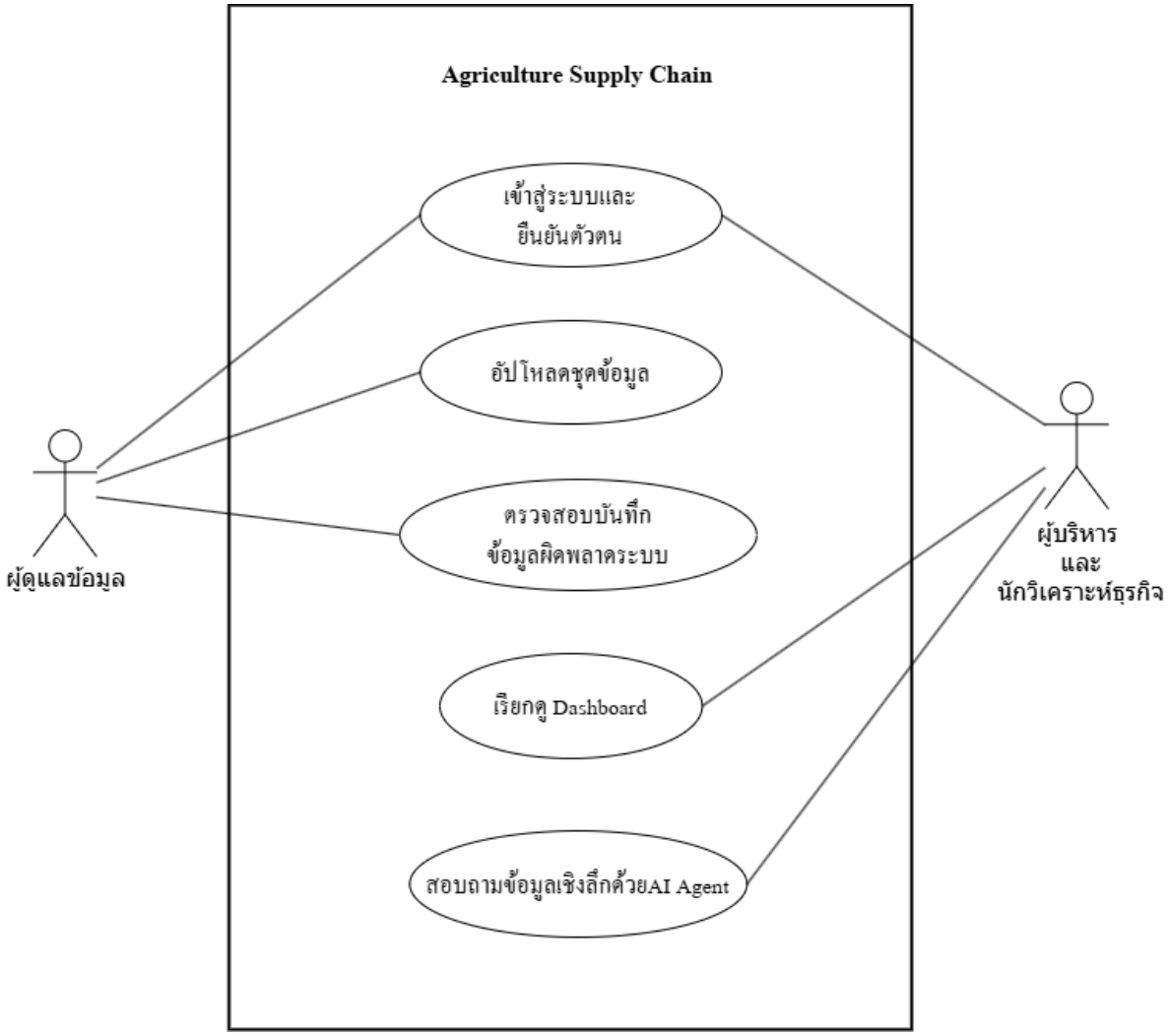
  ขั้นตอนที่ 12-13 (Dashboard Visualization) : ผู้บริหารเข้าสู่ระบบ Web Application ระบบจะส่ง Query ไปยัง Amazon Redshift เพื่อดึงข้อมูลตัวชี้วัด (KPIs) และผลพยากรณ์อนาคต มาสร้างเป็นกราฟิกแสดงผลบนหน้าจอ Dashboard อัตโนมัติ

  ขั้นตอนที่ 14-15 (AI Chat Prompting) : ผู้บริหารมีความต้องการวิเคราะห์ข้อมูลเชิงลึก จึงพิมพ์สอบถามด้วยภาษาธรรมชาติผ่านช่องแชท ระบบ AI Agent บนเซิร์ฟเวอร์ EC2 จะทำการแปลงความหมายเป็นคำสั่งฐานข้อมูล (Text-to-SQL)

  ขั้นตอนที่ 16-17 (Dynamic Response) : เซิร์ฟเวอร์ส่งคำสั่ง SQL ไปดึงข้อมูลที่ Amazon Redshift และนำผลลัพธ์ส่งกลับมาประมวลผลเพื่อแสดงเป็นกราฟหรือข้อความสรุปผลบนหน้าจอแบบเรียลไทม์

3.3. หลักการทำงานของระบบ

Use case Diagram



รูปที่ 3.3. Use case Diagram

3.3.1. การออกแบบหน้าจอผู้ดูแลข้อมูล (Data Administrator Panel)

ส่วนนี้เป็น Web Application ที่รันบน AWS EC2 เพื่อให้ผู้ดูแลระบบเข้ามาจัดการข้อมูล :

- **หน้า Upload Datasets :** มีปุ่มสำหรับเลือกไฟล์ CSV จากแผนกต่าง ๆ เพื่ออัปโหลดเข้าสู่ระบบ
- **หน้า System Logs :** สำหรับตรวจสอบสถานะความสำเร็จหรือผิดพลาดของการส่งข้อมูล

3.3.2. การออกแบบหน้าจอแสดงผลข้อมูลสำหรับผู้บริหาร

- **Executive Summary :** แสดง KPI ที่สำคัญในรูปแบบ Card สรุปยอดรวม
- **Predictive Insights :** แสดงกราฟแนวโน้มอนาคตที่ได้จากการคำนวณของ Machine Learning

3.3.3. การออกแบบหน้าต่างปัญญาประดิษฐ์ (AI Agent Interface)

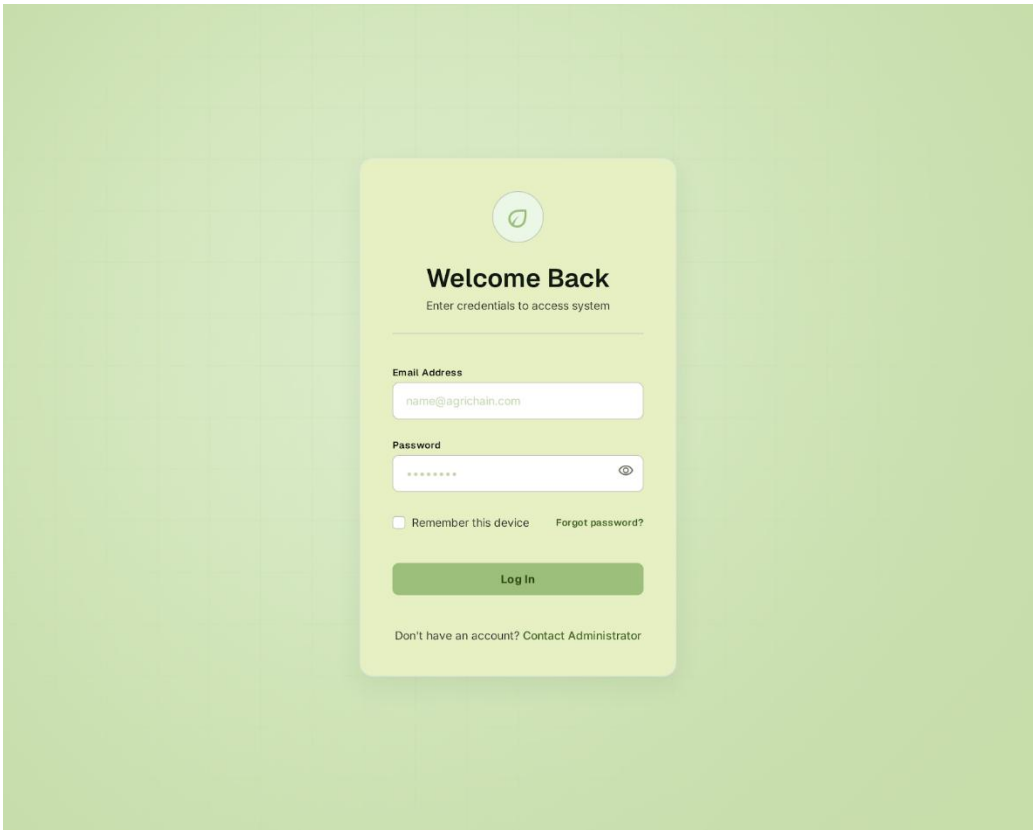
สำหรับให้นักวิเคราะห์ธุรกิจ (Business Analyst) ใช้ตอบคำถามทางธุรกิจในแต่ละแผนก:

- ช่องแชทสำหรับให้นักวิเคราะห์หรือผู้บริหารพิมพ์สอบถามด้วยภาษาปกติ ระบบ AI จะประมวลผลและวาดกราฟิกสรุปผลลัพธ์ (Dynamic Visualization) ตอบกลับบนหน้าจอโดยตรง

3.4. User Interface (Prototype)

3.4.1. หน้าเข้าสู่ระบบ (Authentication & Log-in Page)

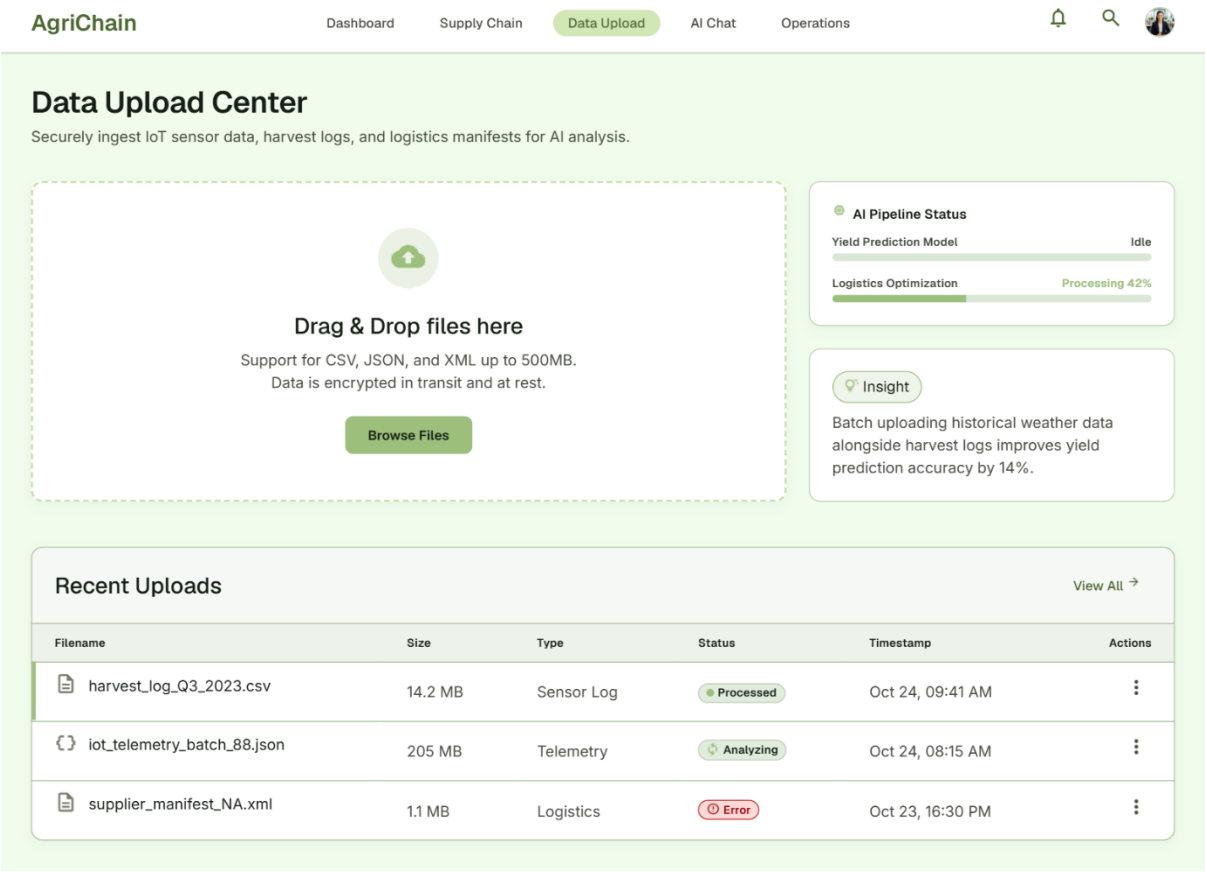
ทำหน้าที่เป็นส่วนหน้าในการยืนยันตัวตนของผู้ใช้งานเพื่อรักษาความปลอดภัยของข้อมูล Agriculture Supply Chain ซึ่งถือเป็นความลับทางธุรกิจที่สำคัญยิ่ง โดยระบบจะบูรณาการร่วมกับกลไก RBAC (Role-Based Access Control) เพื่อตรวจสอบสิทธิ์และจำแนกกลุ่มผู้ใช้งาน (ผู้ดูแลข้อมูล, นักวิเคราะห์, และผู้บริหาร) การออกแบบในลักษณะนี้ช่วยจัดการสิทธิ์การเข้าถึงและปรับแต่งการแสดงผลให้เหมาะสมกับแต่ละบทบาทอย่างมีประสิทธิภาพ ตัวอย่างเช่น การจำกัดสิทธิ์การมองเห็นเมนูศูนย์อัปโหลดข้อมูลสำหรับกลุ่มผู้บริหาร เพื่อป้องกันความเสี่ยงจากความผิดพลาดในการอัปโหลดข้อมูลทับซ้อนโดยไม่ได้ตั้งใจ



รูปที่ 3.4. Authentication & Log-in

3.4.2. ศูนย์อัปโหลดข้อมูล (Data Upload Center)

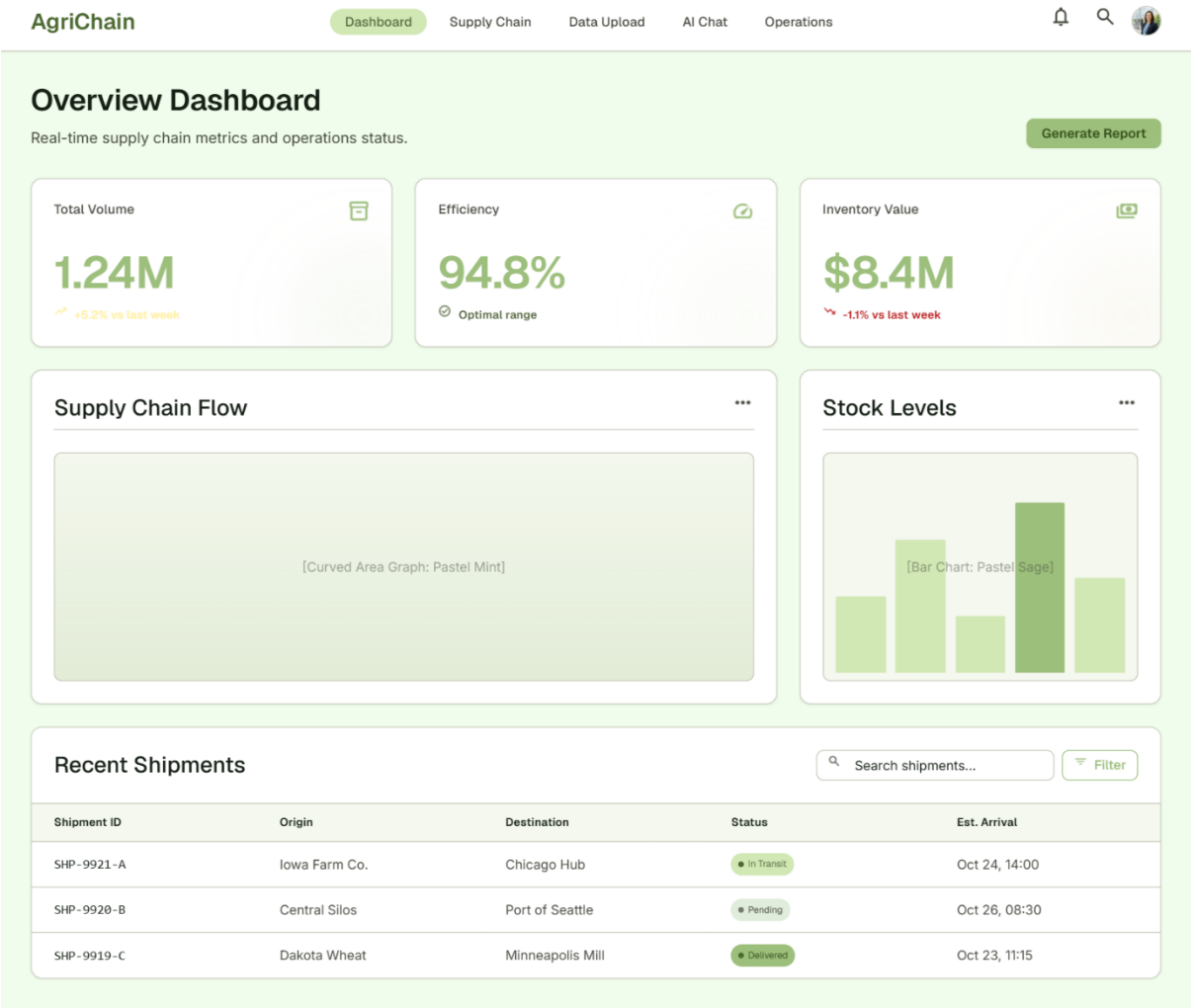
หน้าจอนี้ออกแบบมาเฉพาะสำหรับผู้ดูแลข้อมูล (Data Admin) โดยเน้นความง่ายในการใช้งานผ่านพื้นที่ Drag & Drop สำหรับลากวางไฟล์ CSV พร้อมแสดงประวัติและสถานะการอัปโหลด หน้าจอนี้ทำหน้าที่เป็นจุดเชื่อมต่อ (Gateway) ที่สะดวกและปลอดภัยในการนำข้อมูลดิบระดับปฏิบัติการเข้าสู่พื้นที่จัดเก็บ Amazon S3 ช่วยลดความยุ่งยากโดยที่ผู้ใช้งานไม่ต้องเข้าไปจัดการผ่านหน้าจอที่ซับซ้อนของ AWS Console โดยตรง นอกจากนี้ ทันทีที่การอัปโหลดไฟล์เสร็จสมบูรณ์ ระบบจะส่งสัญญาณกระตุ้นสถาปัตยกรรม Serverless (AWS Step Functions และ Lambda) ให้เริ่มกระบวนการ ETL เพื่อทำความสะอาดและจัดเตรียมข้อมูลโดยอัตโนมัติ



รูปที่ 3.5. Data Upload Center

3.4.3. แผงควบคุมการดำเนินงาน (Visualization Dashboard)

ทำหน้าที่เป็นกระดานแสดงผลหลักที่ดึงข้อมูลจากคลังข้อมูล AWS Redshift มาแสดงผลแบบอัตโนมัติ โดยสรุปตัวชี้วัดสำคัญ (KPIs) ข้ามแผนกไว้ในหน้าเดียว เช่น ปริมาณผลผลิตรวม ประสิทธิภาพโลจิสติกส์ มูลค่าตลาด สถานะคลังสินค้า และตารางการจัดส่งล่าสุด การออกแบบหน้าจอนี้มีจุดประสงค์เพื่อสร้างศูนย์กลางข้อมูลที่ถูกต้องเพียงแหล่งเดียว (Single Source of Truth) ซึ่งช่วยแก้ปัญหาข้อมูลแยกส่วน (Data Silos) ภายในองค์กร ทำให้ผู้บริหารและนักวิเคราะห์ธุรกิจสามารถมองเห็นภาพรวมการดำเนินงานได้แบบเรียลไทม์ และนำข้อมูลไปใช้ประกอบการตัดสินใจเชิงกลยุทธ์ได้อย่างรวดเร็วแม่นยำ

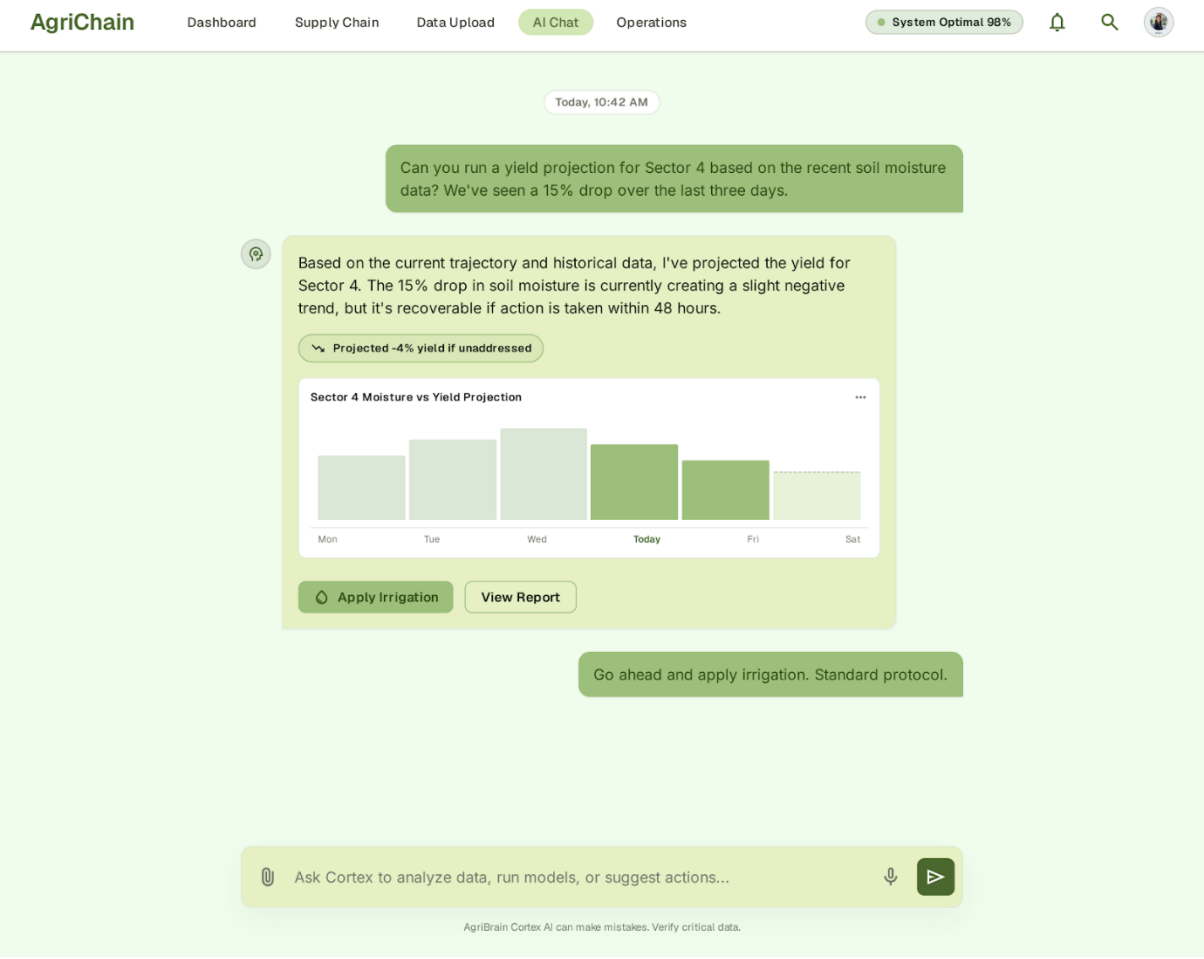


รูปที่ 3.6. Visualization Dashboard

3.4.4. ผู้ช่วย AI อัจฉริยะ (Agentic AI Chat)

เป็นหน้าต่างแชทที่ให้ผู้ใช้งานสามารถพิมพ์สอบถามข้อมูลเชิงลึกทางธุรกิจด้วยภาษาธรรมชาติ (Natural Language) โดย AI Agent จะทำหน้าที่แปลงคำถามเป็นคำสั่งฐานข้อมูล (Text-to-SQL) เพื่อดึงข้อมูลจาก Redshift และประมวลผลออกมาเป็นกราฟสรุปผล (Dynamic Visualization) ตอบกลับบน

หน้าจอโดยตรง หน้าจอนี้ถูกพัฒนาขึ้นเพื่อลดอุปสรรคทางเทคนิคในการเข้าถึงข้อมูล (Democratize Data) ทำให้ผู้บริหารระดับสูงที่อาจไม่มีความเชี่ยวชาญในการเขียนคำสั่ง SQL หรือการตั้งค่าเครื่องมือ BI ที่ซับซ้อน สามารถสั่งการให้ระบบดึงข้อมูลเชิงลึก หรือเรียกดูผลการพยากรณ์จากโมเดล Machine Learning ได้ด้วยตนเองอย่างง่ายดาย



รูปที่ 3.7. Agentic AI Chat

บทที่ 4 ผลการดำเนินงานและระบบต้นแบบ

บทนี้นำเสนอผลการพัฒนาและติดตั้งระบบ AgriChain ซึ่งเป็นแพลตฟอร์ม Data Warehouse สำหรับห่วงโซ่อุปทานสินค้าเกษตรบน Amazon Web Services ประกอบด้วยส่วนติดต่อผู้ใช้งานของเว็บแอปพลิเคชัน ผลการใช้งานบริการต่าง ๆ ของ AWS ที่ติดตั้งจริงบนบัญชี AWS Academy Learner Lab ผลการทดสอบกระบวนการนำเข้าข้อมูล (ETL) และผู้ช่วยวิเคราะห์ข้อมูลด้วยปัญญาประดิษฐ์ (AI Assistant) ปัญหาที่พบระหว่างการติดตั้งและแนวทางแก้ไข รวมถึงข้อจำกัดของระบบและส่วนที่ไม่เป็นไปตามแผนที่วางไว้ในตอนเสนอหัวข้อโครงงาน

4.1. ส่วนติดต่อผู้ใช้งาน

เว็บแอปพลิเคชันพัฒนาด้วย Express (Node.js) ทำหน้าที่ให้บริการหน้าเว็บ ตรวจสอบสิทธิ์ผ่าน DynamoDB รับไฟล์ CSV เพื่อส่งเข้า S3 และส่งต่อคำถามของผู้ใช้ไปยัง AI Agent ที่พัฒนาด้วย FastAPI ซึ่งรันบนเครื่อง EC2 เดียวกัน (พอร์ต 3000 และ 8000 ตามลำดับ) ส่วนติดต่อผู้ใช้งานรองรับภาษาไทยและอังกฤษ ผลการพัฒนาแสดงดังนี้

4.1.1. หน้าเข้าสู่ระบบ (Login)

ผู้ใช้กรอกชื่อผู้ใช้และรหัสผ่าน ระบบตรวจสอบกับตาราง agrichain_user_login ใน DynamoDB โดยเปรียบเทียบรหัสผ่านที่แฮชด้วยอัลกอริทึม scrypt เมื่อผ่านการตรวจสอบจะสร้าง Session token เก็บไว้ในตาราง agrichain_user_session (มี TTL หมดอายุอัตโนมัติ) และคืนค่าเป็น cookie ทุกคำขอ /api/* ที่ไม่ใช่หน้าล็อกอินจะถูกตรวจสอบ Session อีกครั้ง หากไม่ผ่านจะได้รหัส 401 และถูกส่งกลับหน้าล็อกอิน



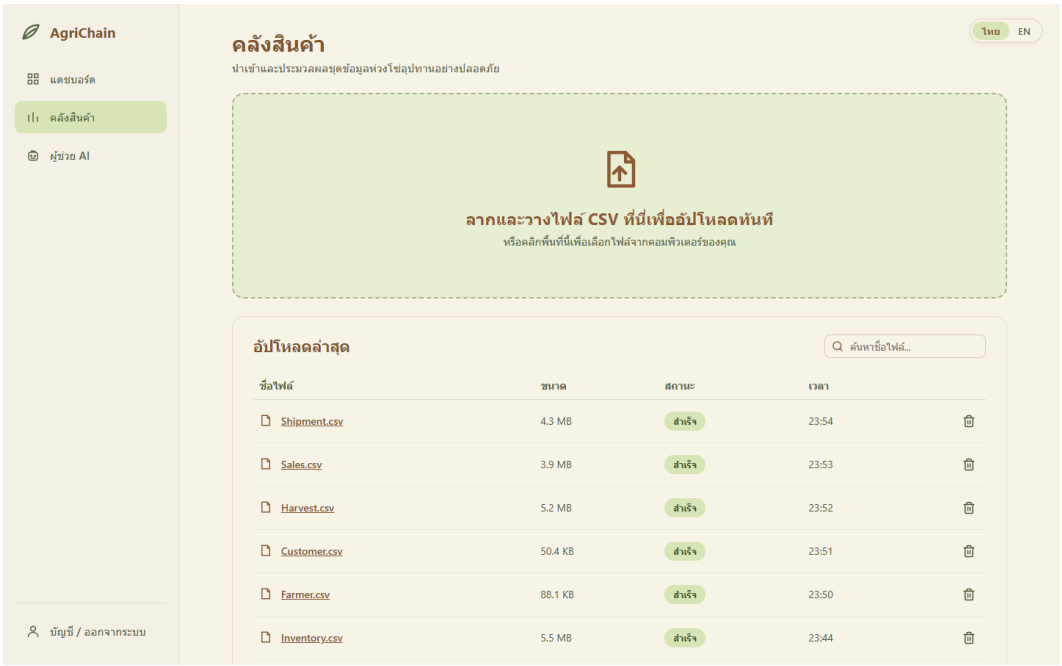
รูปที่ 4.1. หน้าเข้าสู่ระบบ (Login)



รูปที่ 4.2. หน้าเข้าสู่ระบบเมื่อกรอกชื่อผู้ใช้และรหัสผ่าน

4.1.2. หน้าคลังสินค้า (นำเข้าข้อมูล CSV)

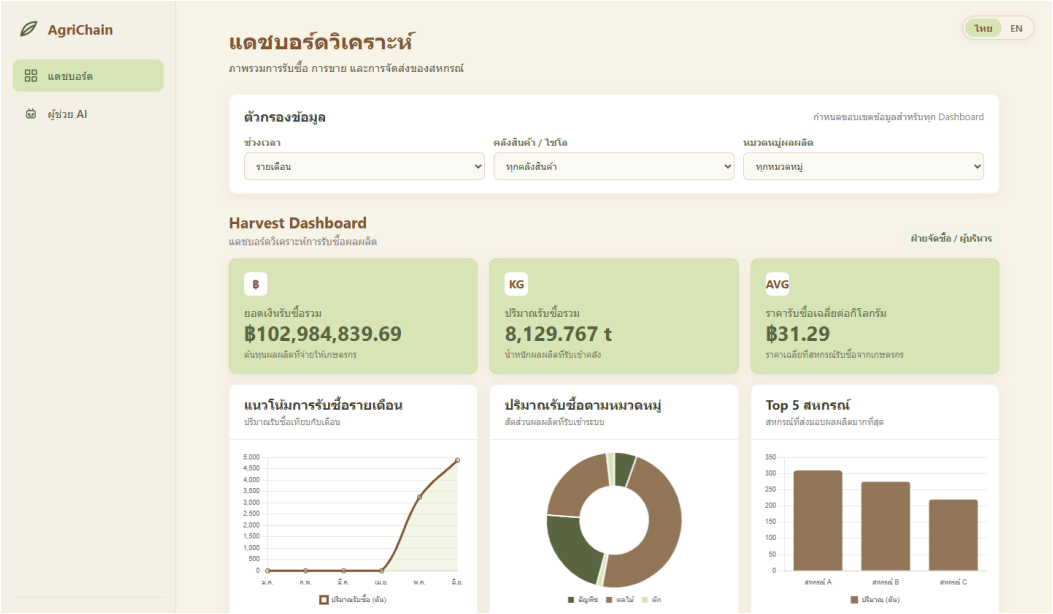
ผู้ดูแลระบบลากไฟล์ CSV มาวางหรือคลิกเพื่อเลือกไฟล์ เว็บจะส่งไฟล์เข้า S3 bucket (โฟลเดอร์ csv-uploads/) และแสดงรายการไฟล์ที่อัปโหลดล่าสุด พร้อมขนาด สถานะและเวลา สามารถค้นหาชื่อไฟล์หรือลบไฟล์ออกจาก S3 ได้ การอัปโหลดแต่ละครั้งจะกระตุ้นกระบวนการ ETL อัตโนมัติ โดยชื่อไฟล์ต้องมีคำว่า Crop, Farmer, Warehouse, Customer, Harvest, Sales, Shipment หรือ Inventory เพื่อให้ระบบเลือกตารางปลายทางได้ถูกต้อง



รูปที่ 4.3. หน้าคลังสินค้า : อัปโหลดไฟล์ CSV และรายการไฟล์ที่นำเข้าแล้ว 8 ไฟล์

4.1.3. Dashboard

เมื่อผู้ใช้เข้าสู่หน้า Dashboard ระบบจะเรียก /api/dashboard จาก FastAPI เพื่อดึงข้อมูลจาก Amazon RDS (PostgreSQL) โดย Backend จะคำนวณข้อมูล KPI จากนั้นส่งข้อมูลกลับมาเป็น JSON ให้ Frontend นำไปแสดงผลเป็น KPI Cards และกราฟด้วย Chart.js โดยข้อมูลจะอ้างอิงจากข้อมูลล่าสุด



รูปที่ 4.4. Dashboard

4.1.4. หน้าผู้ช่วย AI (AI Assistant)

ผู้ใช้พิมพ์คำถามเป็นภาษาไทย ระบบส่งคำถามไปยัง AI Agent ซึ่งใช้โมเดล Qwen (qwen/qwen3.8-27b) ผ่านบริการ Groq แปลงคำถามเป็นคำสั่ง SQL (Text-to-SQL) โดยได้รับคำอธิบาย schema ของคลังข้อมูลเป็นบริบท คำสั่ง SQL ที่ได้ต้องผ่านตัวตรวจสอบ (SQL Guardrail) ก่อนเสมอ ซึ่งอนุญาตเฉพาะคำสั่ง SELECT ตรวจว่าตารางอยู่ใน schema data_warehouse จำกัดจำนวนแถวไม่เกิน 100 แถว และเวลาประมวลผลไม่เกิน 15 วินาที จากนั้นจึงรันบนฐานข้อมูล RDS ผลลัพธ์จะถูกสรุปเป็นภาษาไทย พร้อมกราฟที่เหมาะสมกับชนิดข้อมูล (กราฟแท่งสำหรับการจัดอันดับ กราฟเส้นสำหรับแนวโน้มตามเวลา) และมีปุ่ม "ดูคำสั่ง SQL ที่ใช้ดึงข้อมูล" เพื่อให้ผู้ใช้ตรวจสอบที่มาของตัวเลขได้



รูปที่ 4.5. หน้าผู้ช่วย AI : ตัวอย่างการถามยอดขายสุทธิ ยอดขายรวมตามพืช



รูปที่ 4.6. หน้าผู้ช่วย AI เมื่อเปิดดูคำสั่ง SQL ที่ระบบสร้างและใช้ดึงข้อมูล

4.2. การแบ่งสิทธิ์การเข้าถึงตามบทบาทผู้ใช้ (Role-based Access)

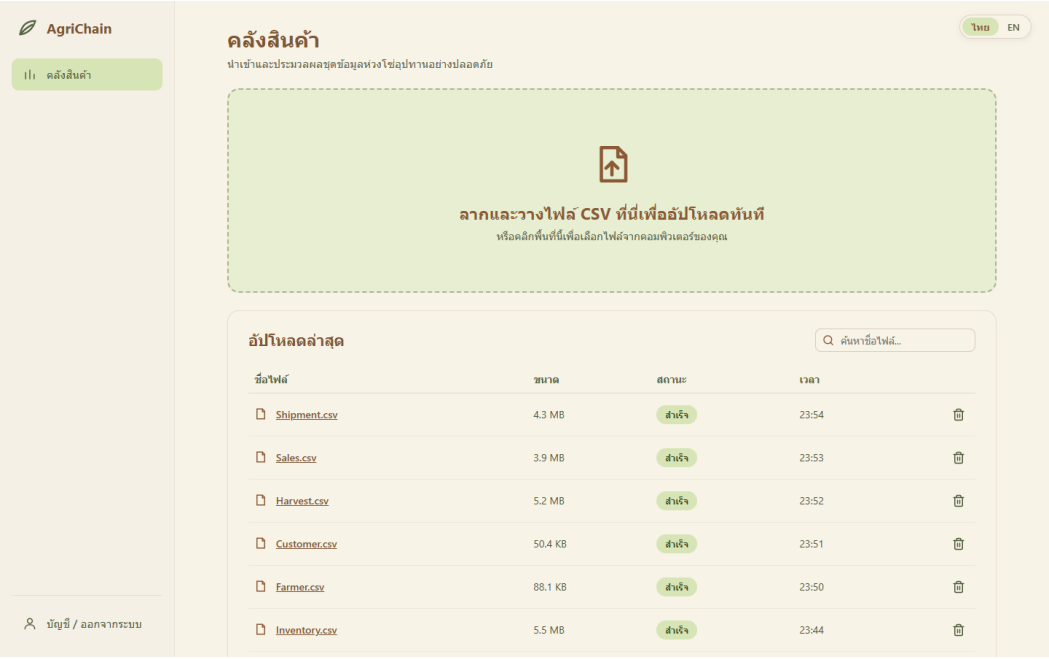
ตามแผนภาพ Use Case ของโครงงาน (รูปที่ 3.3) ระบบมีผู้ใช้ 2 กลุ่ม กลุ่มแรกคือ ผู้ดูแลข้อมูล (Data Admin) มีหน้าที่เข้าสู่ระบบ อัปโหลดชุดข้อมูล และตรวจสอบบันทึกข้อมูลผิดพลาดของระบบ กลุ่มที่สองคือ ผู้บริหารและนักวิเคราะห์ธุรกิจ (Executive และ Data Analyst) มีหน้าที่เข้าสู่ระบบ เรียกดู Dashboard และสอบถามข้อมูลเชิงลึกด้วย AI Agent ระบบจึงกำหนดให้แต่ละบทบาทเข้าถึงคนละหน้า

บทบาทของผู้ใช้เก็บเป็นคุณสมบัติ role ของแต่ละรายการในตาราง agrichain_user_login บน DynamoDB ซึ่งผู้จัดทำสร้างผู้ใช้ทดสอบไว้ 3 บัญชี ตามตารางที่ 4.1 เมื่อล็อกอิน ระบบจะบันทึกบทบาทลงใน Session และใช้ตรวจสิทธิ์ทุกคำขอที่ฝั่งเซิร์ฟเวอร์ 3 ระดับ ได้แก่

1. หน้าเว็บ : หากเปิดหน้าที่บทบาทนั้นไม่มีสิทธิ์ ระบบจะพากลับไปหน้าแรกของบทบาทตนเอง
2. API : คำขอ /api/* ที่ไม่อยู่ในสิทธิ์ของบทบาทจะได้รหัส 403
3. เมนูด้านข้าง : ซ่อนลิงก์ของหน้าที่ไม่มีสิทธิ์ ซึ่งสิทธิ์จริงถูกบังคับที่เซิร์ฟเวอร์

ผู้ใช้	บทบาท (role)	หน้าแรกหลังล็อกอิน	หน้าที่เข้าได้	ใช้งานตาม Use Case
admin01	Data Admin	คลังสินค้า (inventory.html)	คลังสินค้า	อัปโหลดชุดข้อมูล, ดูและลบไฟล์ที่นำเข้า
analyst01	Data Analyst	แดชบอร์ด (dashboard.html)	แดชบอร์ด, ผู้ช่วย AI	เรียกดู Dashboard, สอบถามข้อมูลด้วย AI Agent
exec01	Executive	แดชบอร์ด (dashboard.html)	แดชบอร์ด, ผู้ช่วย AI	เรียกดู Dashboard, สอบถามข้อมูลด้วย AI Agent

ตารางที่ 4.1. ผู้ใช้ทดสอบและสิทธิ์การเข้าถึงหน้าของแต่ละบทบาท



รูปที่ 4.7. มุมมองของ Data Admin (admin01) : เห็นเมนูเฉพาะ "คลังสินค้า"



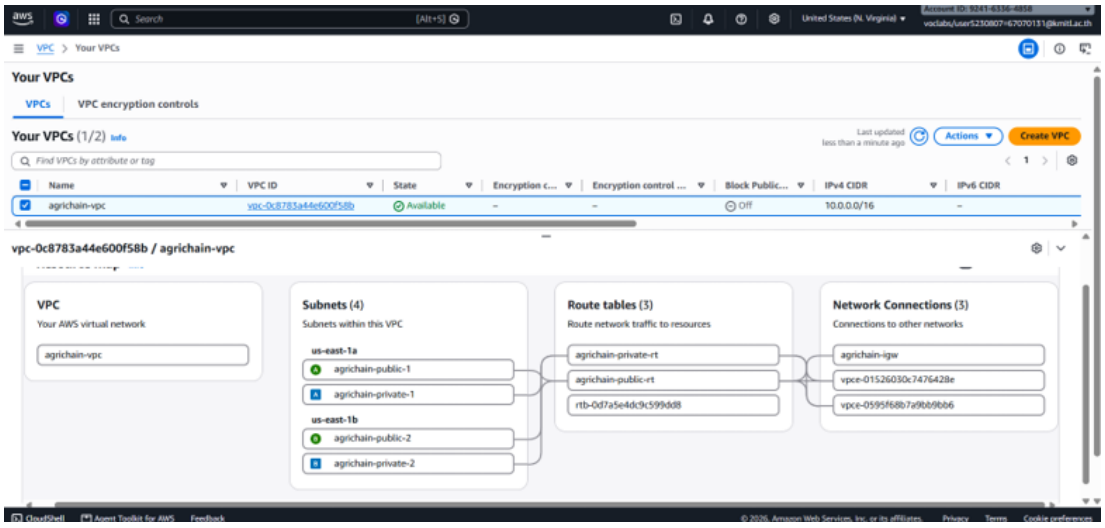
รูปที่ 4.8. มุมมองของ Data Analyst (analyst01) : เห็นเมนู "แดชบอร์ด" และ "ผู้ช่วย AI"

4.3. ผลการดำเนินงานของการใช้งาน AWS ร่วมกับเว็บแอปพลิเคชัน

หัวข้อนี้อธิบายบริการของ AWS ที่ใช้งานจริงทีละบริการ ได้แก่ หน้าที่ในระบบ ค่าที่ตั้ง และผลที่ได้จากการติดตั้ง ภาพประกอบในหัวข้อนี้จัดทำจากภาพที่แคปจาก AWS Management Console

4.3.1. Amazon VPC และเครือข่าย

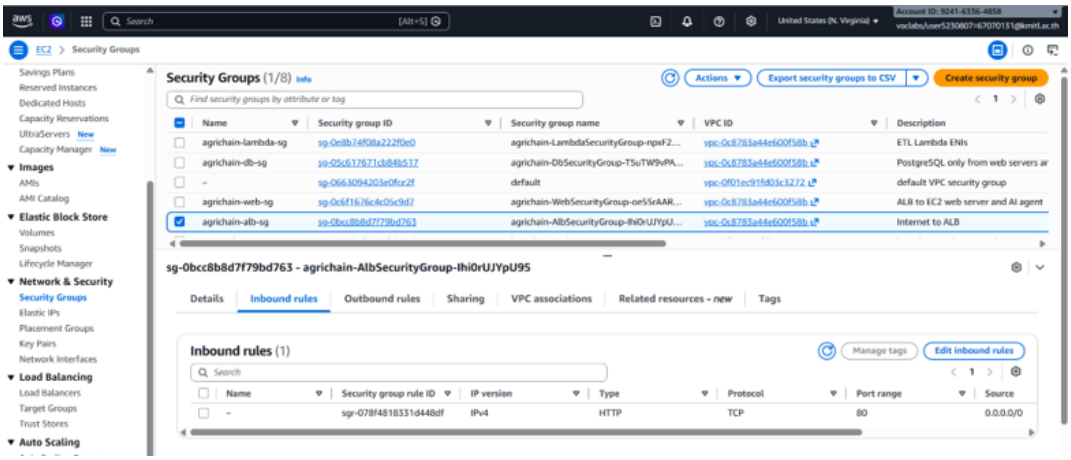
ระบบใช้ VPC ที่สร้างใหม่ (10.0.0.0/16) แทน Default VPC แบ่งเป็น 4 Subnet ใน 2 Availability Zone ได้แก่ Public Subnet 2 แห่ง (10.0.1.0/24 ใน us-east-1a 10.0.2.0/24 ใน us-east-1b) สำหรับ ALB และเครื่อง EC2 และ Private Subnet 2 แห่ง (10.0.11.0/24 และ 10.0.12.0/24) RDS และ Lambda Public Route Table มีเส้นทาง 0.0.0.0/0 ไปยัง Internet Gateway ส่วน Private Route Table ไม่มีเส้นทาง ออกอินเทอร์เน็ต ทรัพยากรใน Private Subnet เข้าถึง S3 และ DynamoDB ผ่าน Gateway VPC Endpoint



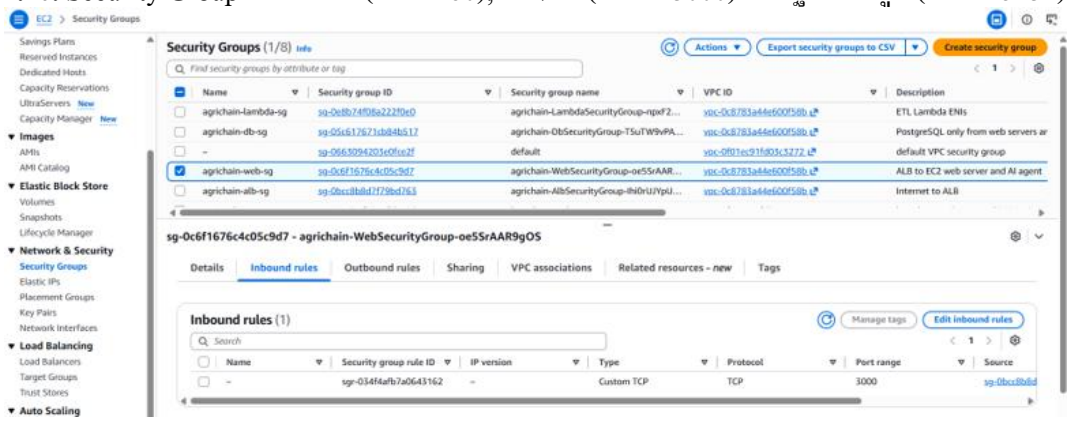
รูปที่ 4.9. VPC agrichain-vpc และแผนผังทรัพยากร (Resource map)

4.3.2. Security Group

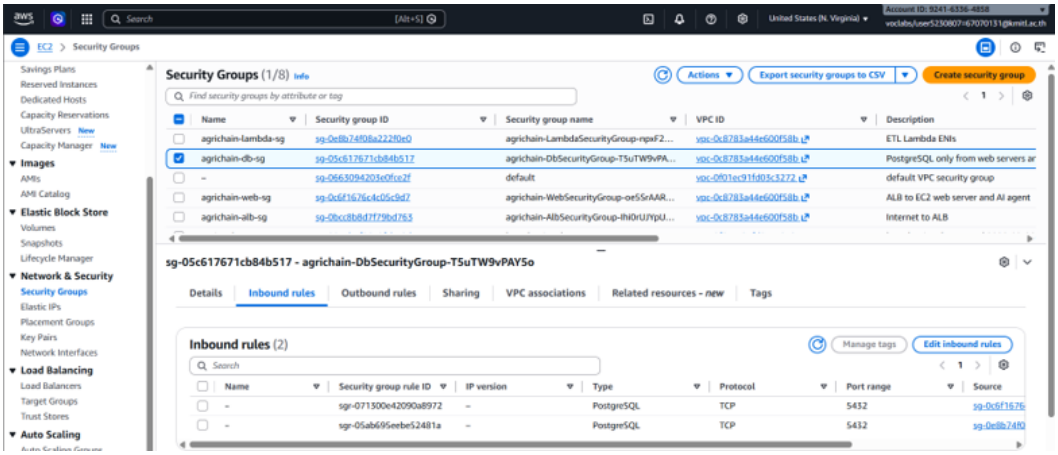
การควบคุมการเข้าถึงกำหนดเป็นลำดับชั้น โดยแต่ละชั้นรับการเชื่อมต่อจากชั้นก่อนหน้าเท่านั้น ได้แก่ ALB รับ HTTP พอร์ต 80 จากอินเทอร์เน็ต, เครื่อง EC2 รับพอร์ต 3000 เฉพาะจาก Security Group ของ ALB, ส่วน RDS รับพอร์ต 5432 เฉพาะ Security Group ของ EC2 และของ Lambda ทำให้ฐานข้อมูลไม่สามารถเข้าถึงจากอินเทอร์เน็ต ข้อมูลผู้ใช้และธุรกิจได้รับการปกป้องระดับเครือข่าย



รูปที่ 4.10. Security Group ของ ALB (พอร์ต 80), เว็บ/AI (พอร์ต 3000) และฐานข้อมูล (พอร์ต 5432) (1/3)



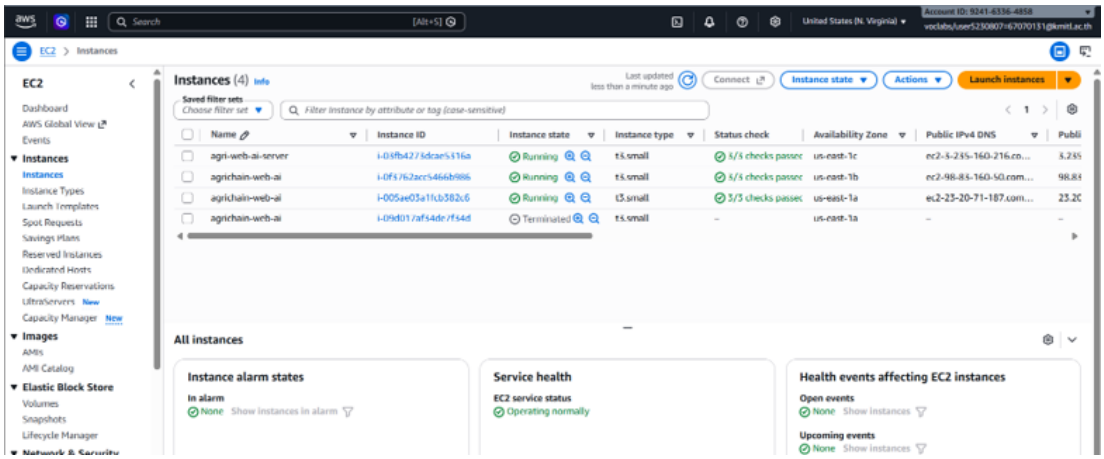
รูปที่ 4.11. Security Group ของ ALB (พอร์ต 80), เว็บ/AI (พอร์ต 3000) และฐานข้อมูล (พอร์ต 5432) (2/3)



รูปที่ 4.12. Security Group ของ ALB (พอร์ต 80), เว็บ/AI (พอร์ต 3000) และฐานข้อมูล (พอร์ต 5432) (3/3)

4.3.3. Amazon EC2 และ Launch Template

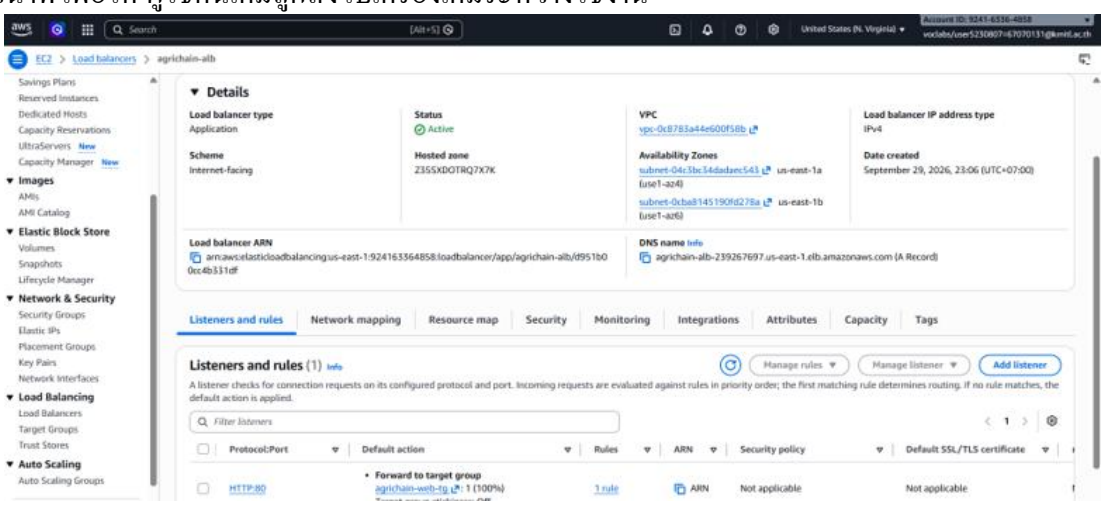
เครื่อง EC2 ใช้ชนิด t3.small ระบบปฏิบัติการ Amazon Linux 2023 กำหนดผ่าน Launch Template และผูก IAM Instance Profile ชื่อ LabInstanceProfile เมื่อเครื่องเริ่มทำงาน User data จะติดตั้ง Node.js และ Python 3.11 ดาวน์โหลด app.zip จาก S3 ติดตั้งไลบรารี อ่านรหัสผ่านฐานข้อมูลและ API key จาก Parameter Store เขียนไฟล์ตั้งค่า .env แล้วตั้งค่า system 2 บริการ ได้แก่ agrichain-web (Express พอร์ต 3000) และ agrichain-ai (FastAPI พอร์ต 8000) เครื่อง EC2 รับหน้าที่เป็นทั้ง เว็บเซิร์ฟเวอร์และ AI Agent ตามที่ออกแบบไว้ และเก็บ Session ไว้ใน DynamoDB เครื่องใดก็ตอบสนอง ผู้ใช้คนเดียวกันได้



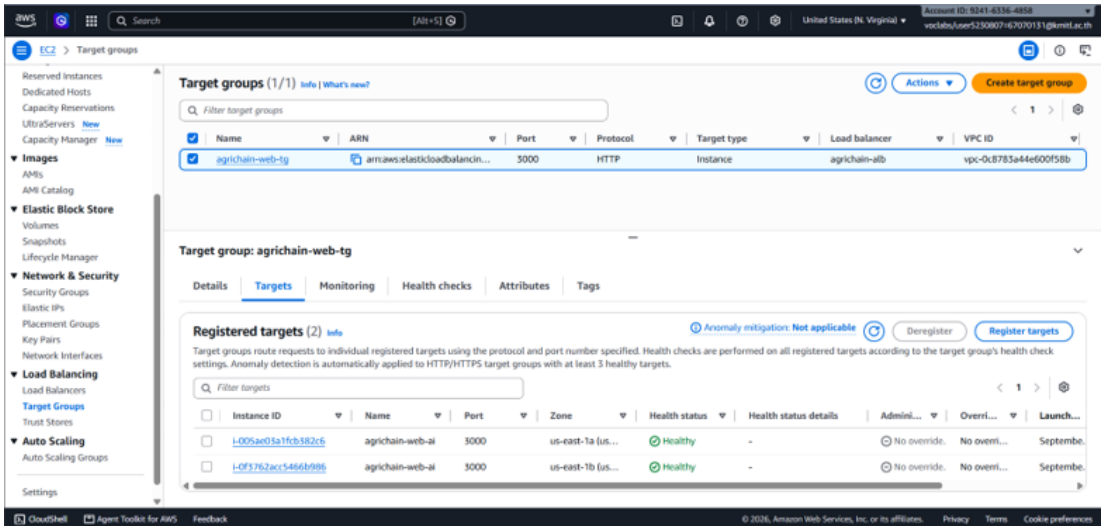
รูปที่ 4.13. เครื่อง EC2 ของระบบ

4.3.4. Elastic Load Balancing (Application Load Balancer)

ใช้ Application Load Balancer แบบ internet-facing กระจาย 2 Availability Zone รับ HTTP พอร์ต 80 แล้วส่งต่อไปยัง Target Group พอร์ต 3000 โดยตรวจสุขภาพของเครื่องที่ path /healthz ทุก 15 วินาที ซึ่ง endpoint นี้ตอบกลับทั้งสถานะของเว็บและสถานะของ AI Agent ALB ส่งคำขอเฉพาะไปยังเครื่องที่สถานะ healthy เท่านั้น และเปิด Sticky session แบบ lb_cookie ที่ระดับ Target Group อายุ 86,400 วินาที เพื่อให้ ผู้ใช้คนเดิมถูกส่งไปเครื่องเดิมระหว่างใช้งาน



รูปที่ 4.14. Application Load Balancer agrichain-alb และ Listener HTTP : 80

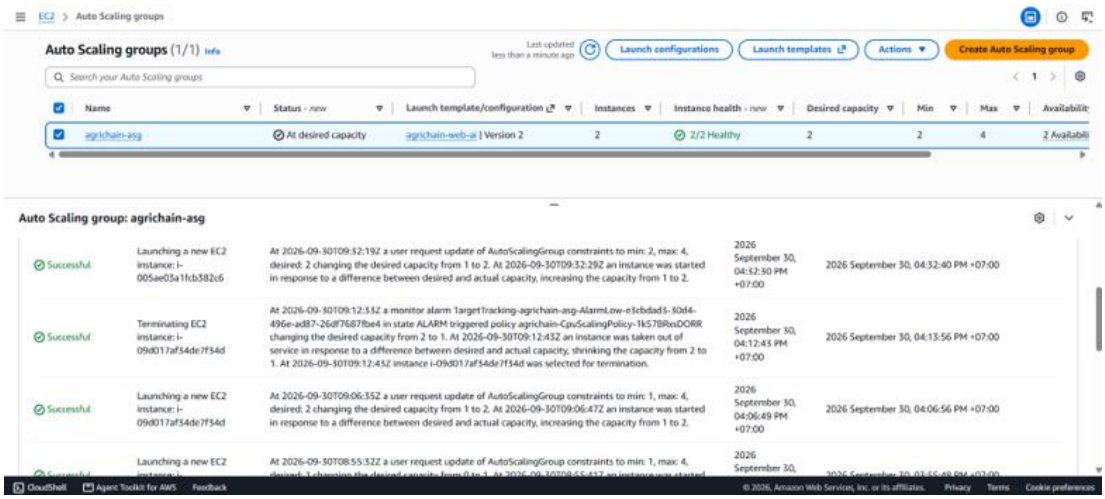


รูปที่ 4.15. สถานะ Health ของเครื่องใน Target Group agrichain-web-tg

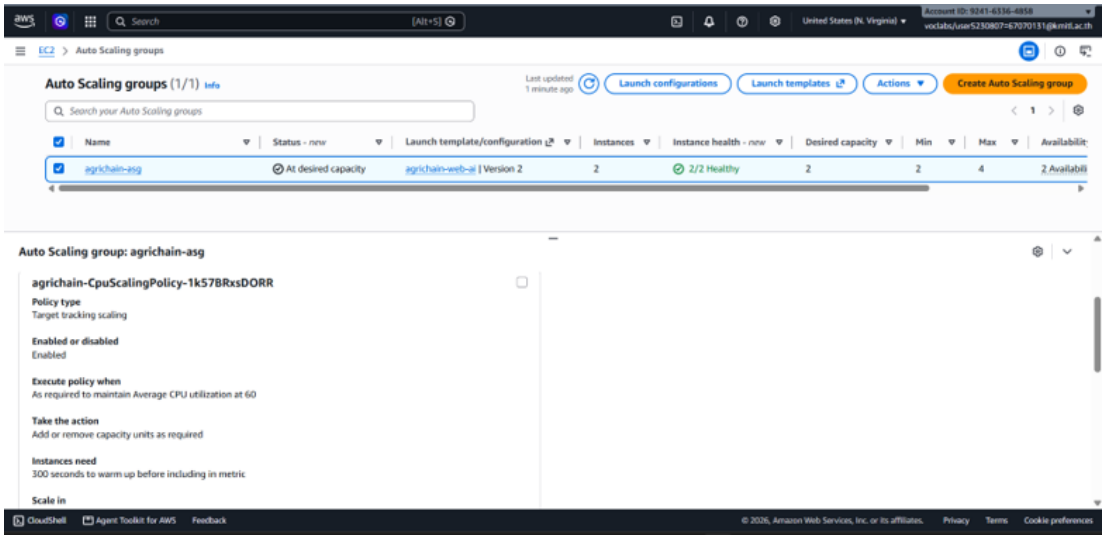
4.3.5. AWS Auto Scaling

Auto Scaling Group ชื่อ agrichain-asg กำหนดจำนวนเครื่องต่ำสุด 1 สูงสุด 4 และกระจายเครื่องอยู่ใน Public Subnet ของทั้ง 2 Availability Zone ใช้การตรวจสุขภาพแบบ ELB เพื่อให้ ALB เป็นผู้ตัดสินว่าเครื่องใดไม่ปกติและสร้างเครื่องใหม่ทดแทน นโยบายการปรับขนาดเป็น Target Tracking โดยใช้เมตริกค่า เฉลี่ย CPU ของกลุ่ม (ASGAverageCPUUtilization) เป้าหมาย 60 เปอร์เซ็นต์ หากใช้งานมากกว่าเป้าหมายจะเพิ่มเครื่อง และเมื่อใช้งานน้อยจะลดเครื่องลงอัตโนมัติ

ประวัติกิจกรรม (Scaling activity history) ในรูปที่ 4.23 แสดงหลักฐานการทำงานจริงของนโยบาย โดยมีหลายครั้งที่ Auto Scaling ตรวจพบว่า CPU ต่ำกว่าเป้าหมาย (AlarmLow) แล้วยกเลิกเครื่องส่วนเกินเอง ส่วนรายการที่ระบุว่า "ผู้ดูแลระบบปรับ desired capacity ด้วยตนเอง" คือครั้งที่ผู้จัดทำเพิ่มจำนวนเครื่องหรือปิด-เปิดระบบเอง ทั้งนี้ค่า Desired/Min ที่แสดงเป็น 2/2 ในรูปเป็นการตั้งชั่วคราวเพื่อสาธิตการกระจายเครื่องข้าม 2 Availability Zone ค่ามาตรฐานของระบบคือต่ำสุด 1 และสูงสุด 4



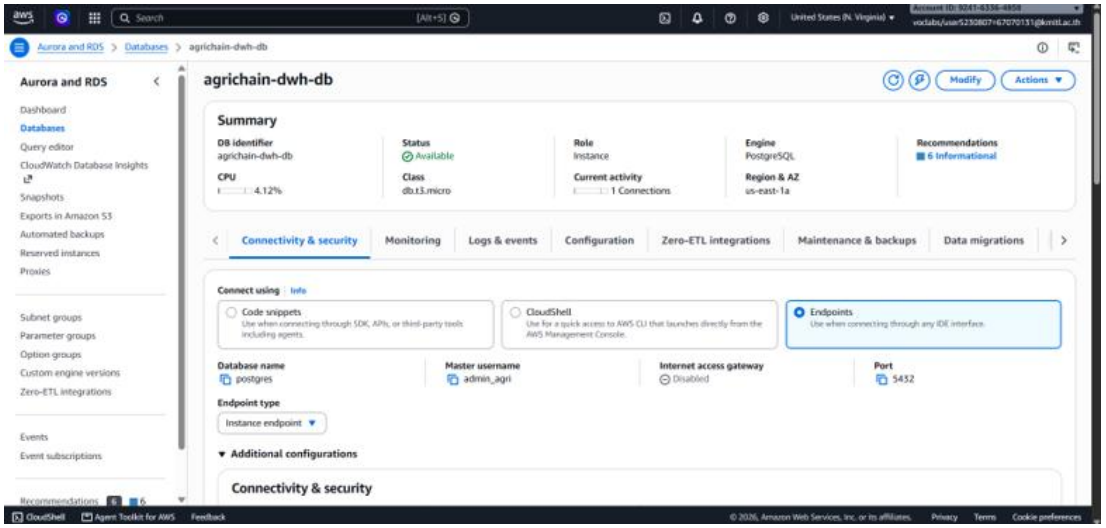
รูปที่ 4.16. Auto Scaling Group agrichain-asg : ประวัติกิจกรรมและนโยบาย Target Tracking (1/2)



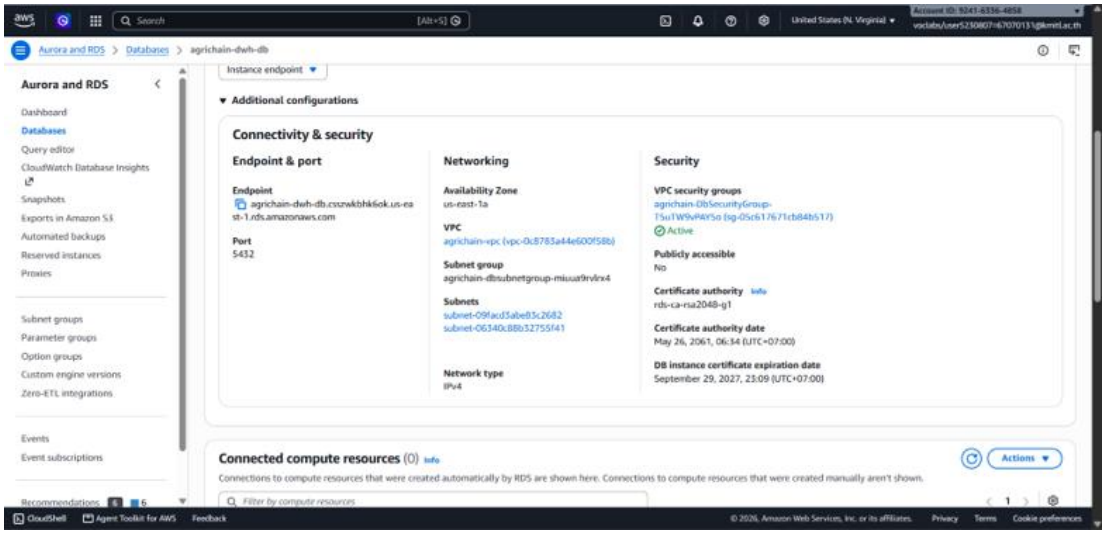
รูปที่ 4.17. Auto Scaling Group agrichain-asg : ประวัติกิจกรรม และนโยบาย Target Tracking (2/2)

4.3.6. Amazon RDS for PostgreSQL

ฐานข้อมูลคลังข้อมูลใช้ Amazon RDS for PostgreSQL (เวอร์ชัน 18.3) ชนิดเครื่อง db.t3.micro พื้นที่ 20 GiB วางอยู่ใน Private Subnet และตั้ง Publicly accessible เป็น false DB Subnet Group ครอบคลุม ทั้ง 2 Availability Zone (Subnet ที่สองสำรองไว้สำหรับ failover) ภายในสร้าง schema data_warehouse ที่ ออกแบบเป็น Star Schema ประกอบด้วยตารางมิติ 5 ตาราง ได้แก่ dim_date, dim_crop, dim_warehouse, dim_farmer, dim_customer และตารางข้อเท็จจริง 4 ตาราง ได้แก่ fact_harvest (การรับซื้อ), fact_sales (การขาย), fact_shipment (การจัดส่ง), fact_inventory (สินค้าคงคลัง) โดยตารางมิติใช้การจัดการการเปลี่ยน แปลงข้อมูลแบบ SCD คือ Type 1 สำหรับพืช, Type 2 สำหรับเกษตรกรและลูกค้า (เก็บ valid_from, valid_to, is_current) และ Type 3 สำหรับคลังสินค้า (เก็บชื่อปัจจุบันและชื่อก่อนหน้า) ตารางข้อเท็จจริง เชื่อมกับตารางมิติด้วย surrogate key



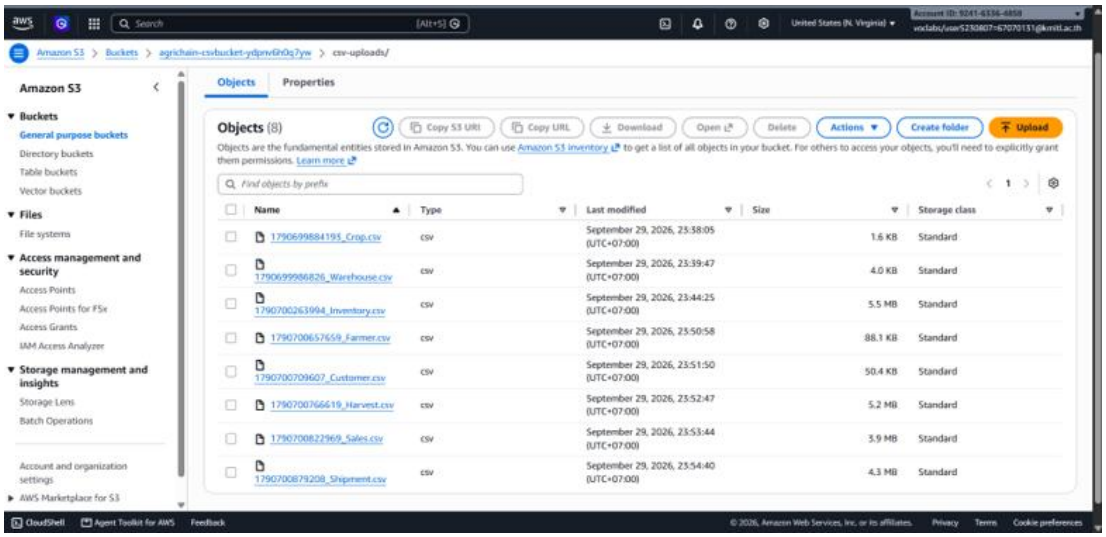
รูปที่ 4.18. ฐานข้อมูล RDS agrichain-dwh-db (Publicly accessible = No) (1/2)



รูปที่ 4.19. ฐานข้อมูล RDS agrichain-dwh-db (Publicly accessible = No) (2/2)

4.3.7. Amazon S3

ใช้ S3 bucket 2 ใบ ใบแรกเก็บไฟล์ CSV ที่ผู้ใช้อัปโหลด (โฟลเดอร์ csv-uploads/) ตั้งค่า Block all public access ครบทั้ง 4 ตัวเลือก และเปิดการส่ง event ไปยัง Amazon EventBridge ใบที่สองเก็บ app.zip และ lambda.zip ที่ใช้ติดตั้งระบบ ไฟล์ที่นำเข้าทั้ง 8 ไฟล์แสดงดังรูป

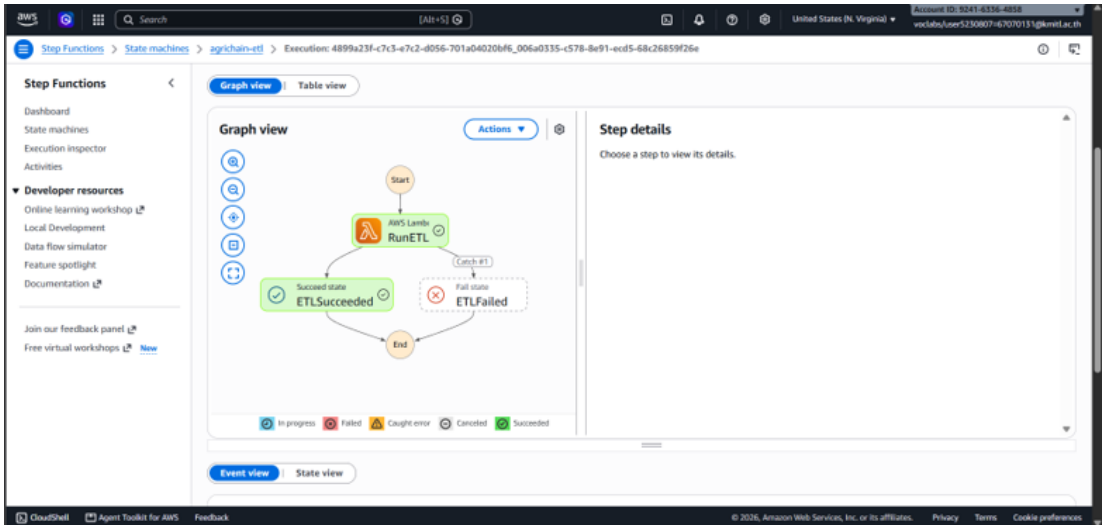


รูปที่ 4.20. ไฟล์ CSV ใน S3 bucket โฟลเดอร์ csv-uploads/

4.3.8. Amazon EventBridge และ AWS Step Functions

เมื่อมีไฟล์ใหม่ถูกสร้างใน S3 จะเกิด event "Object Created" ส่งไปยัง EventBridge rule ชื่อ agrichain-csv-uploaded ซึ่งกรองเฉพาะ bucket ของโครงงานและคีย์ที่ขึ้นต้นด้วย csv-uploads/ แล้วเรียก State Machine ชื่อ agrichain-etl State Machine มีขั้นตอน RunETL (เรียก Lambda) แล้วไปที่ ETLSucceeded เมื่อสำเร็จ หรือไปที่ ETLFailed เมื่อเกิดข้อผิดพลาด มีการตั้ง Retry สูงสุด 3 ครั้งแบบ Exponential backoff เฉพาะข้อผิดพลาดชั่วคราว (เช่น Lambda.ServiceException, OperationalError,

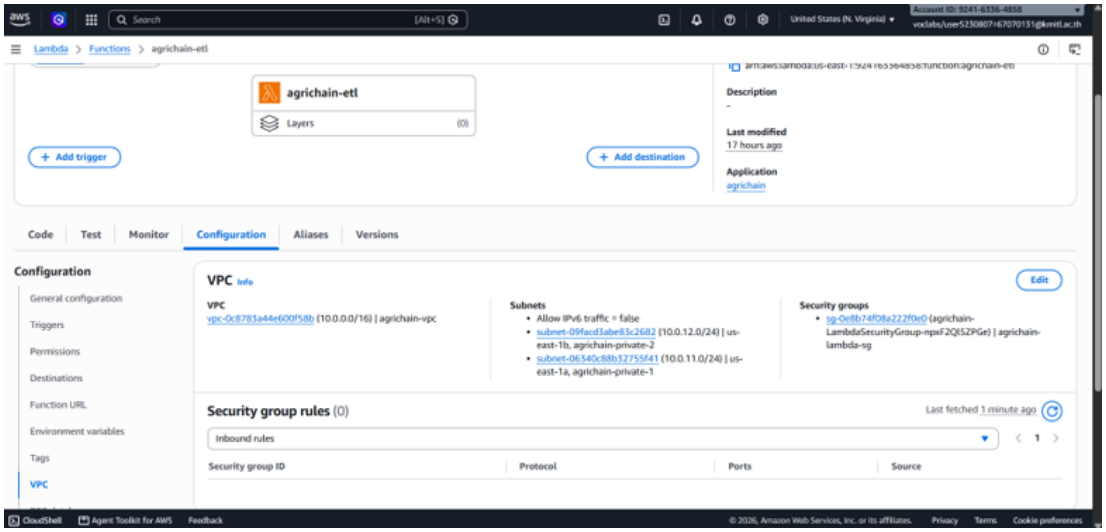
InterfaceError) และตั้ง Timeout 900 วินาที รายการ Execution ในรูปมีทั้งที่สำเร็จ (SUCCEEDED) และล้มเหลว (FAILED) โดยรายการที่ล้มเหลวเกิดจากรอบทดสอบแรกที่ยังมีข้อผิดพลาดของโค้ด ETL หลังแก้ไขแล้วทุกไฟล์ทำงานสำเร็จ



รูปที่ 4.21. Graph view ของ Execution ที่สำเร็จ (RunETL ⟶ ETLSucceeded)

4.3.9. Amazon EventBridge และ AWS Step Functions

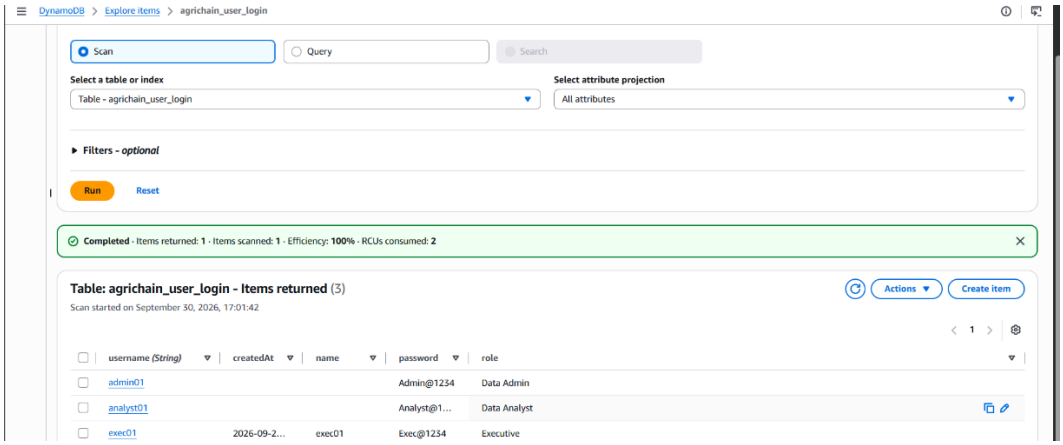
Lambda ชื่อ agrichain-etl ใช้ Python 3.12 หน่วยความจำ 2,048 MB Timeout 900 วินาที ทำงานใน Private Subnet ของ VPC จึงเชื่อมต่อ RDS ได้โดยตรงและเข้าถึง S3 ผ่าน Gateway Endpoint ทำหน้าที่ 2 อย่าง คือ (1) ETL : อ่านไฟล์ CSV จาก S3 เลือกตารางปลายทางจากคำในชื่อไฟล์ ทำความสะอาดข้อมูล (ตัดช่องว่าง แทนค่าว่าง จำกัดค่าติดลบ คำนวณยอดรวม) แปลงรหัสธุรกิจเป็น surrogate key และวันที่เป็น date_key ก่อนโหลดเข้า RDS โดยหยุดพร้อมข้อความชัดเจนหากไฟล์ข้อเท็จจริงอ้างอิงมิติที่ยังไม่ถูกโหลด และ (2) Custom Resource ของ CloudFormation สำหรับสร้าง schema และผู้ใช้ตั้งต้น การเชื่อมต่อฐานข้อมูลเข้ารหัสด้วย SSL



รูปที่ 4.22. การตั้งค่า VPC ของ Lambda agrichain-etl (Private Subnet)

4.3.10. Amazon DynamoDB

ใช้ DynamoDB 3 ตาราง แบบ On-demand (PAY_PER_REQUEST) ได้แก่ agrichain_user_login เก็บบัญชีผู้ใช้และรหัสผ่านที่แฮชด้วย scrypt, agrichain_user_session เก็บ Session token พร้อม TTL ที่ฟิลด์ expiresAt เพื่อลบ Session ที่หมดอายุอัตโนมัติ และ agrichain_chat_history เก็บประวัติการสนทนากับ AI การย้าย Session และประวัติแชตมาไว้ใน DynamoDB ทำให้เว็บเซิร์ฟเวอร์ไม่เก็บสถานะในหน่วยความจำ (Stateless) เครื่อง EC2 จึงเพิ่มลดหรือถูกแทนที่ได้โดยผู้ใช้ไม่ต้องล็อกอินใหม่



รูปที่ 4.23. ตาราง DynamoDB

4.4. ผลการทดสอบระบบ

4.4.1. ผลการนำเข้าข้อมูล (ETL)

ทดสอบโดยอัปโหลดไฟล์ CSV ทั้ง 8 ไฟล์ตามลำดับที่กำหนดผ่านหน้าคลังสินค้า ทุกไฟล์ผ่านกระบวนการ Step Functions และ Lambda สำเร็จ โหลดข้อมูลรวม 170,504 แถวเข้าสู่ schema

ลำดับ	ไฟล์	ตารางปลายทาง	จำนวนแถวที่โหลด	ผล
1	Crop.csv	dim_crop	15	สำเร็จ
2	Farmer.csv	dim_farmer	300	สำเร็จ
3	Warehouse.csv	dim_warehouse	15	สำเร็จ
4	Customer.csv	dim_customer	200	สำเร็จ
5	Harvest.csv	fact_harvest	50,000	สำเร็จ
6	Sales.csv	fact_sales	30,000	สำเร็จ
7	Shipment.csv	fact_shipment	25,000	สำเร็จ
8	Inventory.csv	fact_inventory	64,974	สำเร็จ
	รวม		170,504	

ตารางที่ 4.2. ผลการนำเข้าไฟล์ CSV ด้วย ETL

4.4.2. ผลการทดสอบผู้ช่วย AI

ทดสอบด้วยคำถามภาษาไทย 3 ข้อผ่านหน้าผู้ช่วย AI ผลลัพธ์ที่ได้จากฐานข้อมูล RDS แสดงใน ตารางที่ 4.3. คำสั่ง SQL ที่ระบบสร้างผ่านตัวตรวจสอบ Guardrail ทุกข้อ และตัวเลขในคำตอบมาจาก ผลลัพธ์ของฐานข้อมูลโดยตรง ไม่ได้ถูกสร้างขึ้นโดยโมเดล

คำถาม	ตารางที่ใช้	ผลลัพธ์	รูปแบบการแสดงผล
ยอดขายสุทธิทั้งหมดเท่าไร	fact_sales	2,958,202,618.86 บาท	ข้อความสรุป
ยอดขายรวมแยกตามพืช 5 อันดับแรก	fact_sales, dim_crop	1) กาแฟ 916,831,465.06 2) ทุเรียน 522,228,574.62 3) มังคุด 233,720,481.82 4) ลิ้นจี่ 227,084,772.53 5) ข้าวหอมมะลิ 221,326,490.21 (บาท)	ข้อความ + กราฟแท่ง
ยอดรับซื้อผลผลิตรวมแยกตามปี	fact_harvest, dim_date	ปี 2021 : 1,587,294,395.78 ปี 2022 : 1,553,500,220.03 ปี 2023 : 1,549,303,223.34 ปี 2024 : 1,518,597,712.34 ปี 2025 : 1,547,229,290.78 (บาท)	ข้อความ + กราฟเส้น

ตารางที่ 4.3. ผลการทดสอบผู้ช่วย AI

4.4.3. ผลการทดสอบความพร้อมใช้งาน

- **การตรวจสุขภาพ :** ตรวจสอบ /healthz ผ่าน ALB ได้ผล {"status":"ok","ai":"ok"} ทั้งเว็บ Express และ AI Agent FastAPI พร้อมทำงาน
- **การกระจายข้าม AZ :** เมื่อเพิ่มจำนวนเครื่องเป็น 2 เครื่อง ALB ลงทะเบียนเป้าหมายใหม่และเปลี่ยนสถานะเป็น healthy ได้หลังเครื่องติดตั้งเสร็จ (ประมาณ 5–8 นาที) เครื่องทั้งสองอยู่คนละ Availability Zone (us-east-1a และ us-east-1b)
- **การปรับขนาดอัตโนมัติ :** นโยบาย Target Tracking ลดจำนวนเครื่องอัตโนมัติเมื่อ CPU ต่ำกว่าเป้าหมาย และ ALB ตัดเครื่องที่ถูกยกเลิกออกจาก Target Group โดยไม่กระทบการ ใช้งาน
- **การหยุดและเปิดระบบ :** หลังปิดและเปิดระบบใหม่ (หยุด RDS และตั้ง Auto Scaling เป็น 0 แล้วเปิดกลับ) ข้อมูลที่โหลดไว้ยังอยู่ครบ และผู้ใช้กลับมาล็อกอินและสอบถาม AI ได้ตามปกติ เนื่องจากข้อมูลอยู่ใน RDS, S3 และ DynamoDB ซึ่งแยกจากเครื่อง EC2

4.4.4. ข้อจำกัดของระบบและส่วนที่ไม่เป็นไปตามแผน

เนื่องจากการติดตั้งใช้บัญชี AWS Academy Learner Lab ซึ่งมีข้อจำกัดด้านสิทธิ์ บริการที่ใช้ได้ และงบประมาณ รวมถึงการปรับแนวทางระหว่างพัฒนา ทำให้บางส่วนแตกต่างจากที่เสนอไว้ในหัวข้อโครงงาน ตารางที่ 3.1สรุปความแตกต่างระหว่างแผนกับสิ่งที่ทำจริง พร้อมเหตุผลและผลกระทบ

ที่	สิ่งที่วางแผนไว้	สิ่งที่ทำจริง	เหตุผล	ผลกระทบ
1	คลังข้อมูลบน Redshift (ตามสถาปัตยกรรม)	Amazon RDS for PostgreSQL ใน Private Subnet	Learner Lab ไม่อนุญาตให้ใช้ Amazon Redshift (ในบัญชีมีฟังก์ชันของ Lab ที่ปิด Redshift อัตโนมัติ) และ RDS PostgreSQL ตรงกับขอบเขตที่ระบุในรายงานเสนอหัวข้อ	รองรับข้อมูลขนาดหลักแสนแถวได้ดี แต่ไม่ใช่ฐานข้อมูลแบบคอลัมน์ ประสิทธิภาพกับข้อมูลระดับหลายร้อยล้านแถวจะจำกัดกว่า Redshift
2	RDS แบบ Multi-AZ เพื่อพร้อมใช้งานสูง	RDS แบบ Single-AZ (DBMultiAZ = false) โดย DB Subnet Group ครอบคลุม 2 AZ	Multi-AZ ทำให้ต้นทุนเพิ่มเป็นสองเท่าและมักใช้ไม่ได้กับรุ่นทดลอง/free tier ของบัญชี Lab จึงปิดไว้ตามค่าเริ่มต้นของเทมเพลต	ไม่มี automatic failover ของฐานข้อมูล หาก AZ ที่ RDS อยู่ขัดข้อง ระบบวิเคราะห์จะใช้งานไม่ได้จนกว่าจะกู้คืน
3	เข้ารหัสเชื่อมต่อด้วย HTTPS	ALB รับเฉพาะ HTTP พอร์ต 80	ต้องมีชื่อโดเมนและใบรับรองจาก AWS Certificate Manager ซึ่งอยู่นอกขอบเขตและงบประมาณของ Lab	ข้อมูลระหว่างเบราว์เซอร์กับ ALB ไม่ถูกเข้ารหัส เหมาะกับต้นแบบ ไม่เหมาะกับใช้งานจริง (RDS และ Lambda ใช้ SSL)
4	เครือข่ายที่ปลอดภัยสูงสุด (เครื่อง EC2	เครื่อง EC2 อยู่ใน Public Subnet	NAT Gateway มีค่าใช้จ่ายรายชั่วโมงสูง และเครื่อง EC2 ต้องเรียก API ของ	เครื่อง EC2 มีที่อยู่ Public IP แม้ถูกจำกัดด้วย Security Group

	อยู่ Private Subnet ผ่าน NAT Gateway)	และไม่มี NAT Gateway	Groq บนอินเทอร์เน็ต จึงให้มี Public IP แต่ปิดทุกพอร์ต ยกเว้นพอร์ต 3000 จาก ALB	
5	สร้าง IAM Role เฉพาะงานตามหลักสิทธิ์ขั้นต่ำ	ใช้ LabRole และ LabInstance Profile ที่ Lab จัดให้ทุกบริการ	บัญชี Learner Lab ไม่อนุญาตให้สร้าง IAM Role ใหม่	สิทธิ์ของ EC2, Lambda และ Step Functions กว้างกว่าที่จำเป็น
6	S3 เรียก Step Functions โดยตรงตามลูกศร Event Trigger ในแผนภาพ	S3 ส่ง event ไป EventBridge แล้ว Event Bridge เรียก Step Functions	S3 ไม่สามารถเริ่ม State Machine ได้โดยตรง ต้องมีตัวกลาง	มีส่วนประกอบเพิ่มแต่ผลลัพธ์การทำงานเหมือนที่ออกแบบ
7	ETL ทำ SCD Type 2 เต็มรูปแบบ	เพิ่มแถวใหม่ valid_from, valid_to และ is_current แต่ไม่ปิด valid_to แถวเวอร์ชันเก่า และโหลดแบบ append เท่านั้น	ข้อมูลตัวอย่างที่ทดสอบโหลดครั้งแรก ไม่มีข้อมูลเปลี่ยนแปลงย้อนหลัง	การอัปโหลดไฟล์มิติเดิมซ้ำจะทำให้ได้แถวซ้ำ และไฟล์ข้อเท็จจริงเดิมซ้ำจะถูกปฏิเสธเพราะคีย์ซ้ำ

ตารางที่ 4.4. ข้อจำกัดและส่วนที่ไม่เป็นไปตามแผน

ข้อจำกัดอื่น ๆ ของระบบ

- **การพึ่งพาบริการภายนอก :** ผู้ช่วย AI พึ่งพาบริการ Groq ผ่านอินเทอร์เน็ต หากบริการขัดข้องหรือ API key หมดสิทธิ์ ระบบจะตอบคำถามเชิงวิเคราะห์ไม่ได้ แต่หน้านำเข้าข้อมูลและล็อกอินใช้ได้
- **ความเป็นส่วนตัวของข้อมูล :** คำถามที่ผู้ใช้พิมพ์และโครงสร้างตาราง (schema) ถูกส่งไปยัง LLM เพื่อสร้าง SQL และผลลัพธ์แบบสรุป (จำกัดไม่เกิน 100 แถว) ถูกส่งไปสรุปเป็นภาษาไทย ข้อมูลระดับบุคคลจึงถูกตัดออกจากคลังข้อมูลตั้งแต่ขั้น ETL

- **ความถูกต้องของ Text-to-SQL :** โมเดลอาจสร้าง SQL ที่ไม่ตรงเจตนา แม้ผ่าน Guardrail ผู้ใช้จึงควรตรวจสอบ SQL ที่แสดงประกอบคำตอบเมื่อใช้ตัวเลขประกอบการตัดสินใจ และคำตอบของบางคำถามอาจใช้เวลานานเนื่องจากต้องเรียกโมเดลสองครั้ง (สร้าง SQL และสรุปผล)
- **การจัดการผู้ใช้และสิทธิ์ :** บัญชีผู้ใช้และบทบาทถูกสร้างเป็นรายการใน DynamoDB โดยตรง ยังไม่มีหน้าเว็บสำหรับจัดการผู้ใช้หรือเปลี่ยนบทบาท สิทธิ์แบ่งตามหน้าเว็บ 3 บทบาทเท่านั้น
- **การปล่อยเวอร์ชันใหม่ :** เมื่ออัปเดตโค้ดและติดตั้งซ้ำ เครื่อง EC2 เดิมไม่ถูกเปลี่ยนอัตโนมัติต้องยกเลิกเครื่องเองหรือสั่ง Instance Refresh จึงจะใช้โค้ดใหม่

บทที่ 5 สรุปผลการดำเนินงาน

บทนี้สรุปผลการพัฒนาและติดตั้งระบบ AgriChain เทียบกับวัตถุประสงค์และขอบเขตที่เสนอไว้ ประโยชน์ที่ได้รับ ปัญหาและอุปสรรค และข้อจำกัดของระบบ จากนั้นประมาณการค่าใช้จ่ายของระบบบน AWS อย่างละเอียด โดยใช้ทั้งข้อมูลการใช้งานจริงของบัญชี (จาก AWS Cost Explorer) และราคาทางการของ AWS พร้อมเปรียบเทียบสถานการณ์การใช้งานแบบต่าง ๆ และเสนอแนวทางลดค่าใช้จ่าย รวมทั้งการวิเคราะห์และออกแบบเพิ่มเติมโดยไม่คำนึงถึงค่าใช้จ่าย และข้อเสนอแนะสำหรับการพัฒนาต่อ

5.1. ที่มาและการทำงานโดยรวมของระบบ

สหกรณ์การเกษตรทำหน้าที่เป็นศูนย์กลางรับซื้อและจัดจำหน่ายผลผลิตทางการเกษตร ข้อมูลของแต่ละขั้นตอน ได้แก่ การรับซื้อจากเกษตรกร การจัดเก็บในคลังสินค้า การจัดส่ง และการขาย มักถูกเก็บแยกกันในแต่ละแผนก ทำให้ฝ่ายบริหารไม่เห็นภาพรวมของธุรกิจและตัดสินใจได้ช้า โครงงานนี้จึงพัฒนาแพลตฟอร์มคลังข้อมูล (Data Warehouse) แบบรวมศูนย์ชื่อ AgriChain บน Amazon Web Services ที่นำข้อมูลจากไฟล์ CSV ของแต่ละแผนกมาทำความสะอาด แปลง และโหลดเข้าฐานข้อมูลเชิงวิเคราะห์แบบ Star Schema แล้วให้ผู้บริหารและนักวิเคราะห์ดูแดชบอร์ดและสอบถามข้อมูลเป็นภาษาไทยผ่านผู้ช่วย AI

การทำงานโดยรวมของระบบแบ่งเป็น 3 ส่วนหลัก ส่วนแรกคือส่วนนำเข้าข้อมูล เมื่อผู้ดูแลข้อมูลอัปโหลดไฟล์ CSV ผ่านหน้าเว็บ ไฟล์จะถูกเก็บใน Amazon S3 ซึ่งส่ง event ไปยัง Amazon EventBridge เพื่อเริ่ม AWS Step Functions และเรียก AWS Lambda ที่ทำ ETL โหลดข้อมูลเข้า Amazon RDS for PostgreSQL ส่วนที่สองคือส่วนให้บริการ เว็บเซิร์ฟเวอร์ Express และ AI Agent แบบ FastAPI รันบนเครื่อง EC2 ใน Auto Scaling Group 2 Availability Zone ภายใต้ Application Load Balancer โดยเก็บบัญชีผู้ใช้ Session และประวัติแชตใน Amazon DynamoDB ส่วนที่สามคือส่วนวิเคราะห์ AI Agent ใช้โมเดล Qwen ผ่านบริการ Groq แปลงคำถามภาษาไทยเป็น SQL ผ่านตัวตรวจสอบ ที่อนุญาตเฉพาะคำสั่งอ่านข้อมูล แล้วสรุปผลพร้อมกราฟ ผู้ใช้แต่ละบทบาทเห็นคนละหน้าตาม Use Case

ผลลัพธ์ที่ได้จากการดำเนินงานสรุปเป็นตัวเลขได้ดังนี้ โหลดข้อมูลทดสอบ 8 ไฟล์ รวม 170,504 แถว เข้าตาราง 9 ตารางใน schema data_warehouse ผู้ช่วย AI ตอบคำถามเชิงวิเคราะห์ได้พร้อมกราฟ และระบบรองรับผู้ใช้ 3 บทบาท (Data Admin, Data Analyst และ Executive) ชุดข้อมูลที่ใช้ทดสอบเป็นข้อมูลจำลองที่สร้างด้วยสคริปต์ generate_dataset.py ครอบคลุมช่วงปี 2021–2025 รายละเอียดอยู่ในภาคผนวก ช

5.2. ผลการดำเนินงานเทียบกับวัตถุประสงค์และขอบเขต

ตารางที่ 5.1 เทียบผลที่ได้กับวัตถุประสงค์ 4 ข้อ และตารางที่ 5.2 เทียบกับขอบเขตของโครงงานที่เสนอไว้ โดยระบุว่าทำได้ครบ ทำได้บางส่วน หรือไม่ได้ทำ

วัตถุประสงค์	ผลที่ได้	สถานะ
1.ให้ผู้บริหารและนักวิเคราะห์มีศูนย์กลางข้อมูลดูภาพรวมได้จากที่เดียว	คลังข้อมูล Star Schema บน RDS รวมข้อมูลการรับซื้อ การขาย การจัดส่ง และสินค้าคงคลัง 170,504 แถว เข้าถึงผ่านเว็บ	บรรลุ
2. ให้ติดตามตัวชี้วัด (KPI) ผ่าน Business Dashboard เพื่อตัดสินใจเชิงกลยุทธ์	แดชบอร์ดแสดงยอดเงินรับซื้อ ปริมาณ ราคาเฉลี่ย แนวโน้มรายเดือน สัดส่วนหมวดหมู่ และ Top 5 พร้อมตัวกรอง และมีผู้ช่วย AI ตอบคำถามเชิงลึก	บรรลุบางส่วน
3.องค์กรสร้างระบบจัดการข้อมูลขนาดใหญ่ได้โดยไม่ต้องซื้อฮาร์ดแวร์เซิร์ฟเวอร์	ติดตั้งทั้งระบบบน AWS ค่าใช้จ่ายจริงของโครงงานถึงวันที่จัดทำรายงานประมาณ $2.45 (หัวข้อ 5.6)ไม่มีการลงทุนฮาร์ดแวร์	บรรลุ
4. ให้ระบบยืดหยุ่น ขยายตัวได้ และพร้อมใช้งาน	Auto Scaling Group (ต่ำสุด 1 สูงสุด 4) กระจาย 2 AZ ภายใต้ ALB ที่ตรวจสุขภาพเครื่อง และทดสอบการลดเครื่องอัตโนมัติสำเร็จ แต่ฐานข้อมูลเป็น Single-AZ และยังไม่ได้ทดสอบ scale-out ใต้ภาระงานสูง	บรรลุบางส่วน

ตารางที่ 5.1. ผลการดำเนินงานเทียบกับวัตถุประสงค์

ขอบเขตที่เสนอ	สิ่งที่ทำจริง	สถานะ
คลังข้อมูล Star Schema (Fact: การรับซื้อ การขาย การจัดส่ง และ Dimension)	Fact 4 ตาราง Dimension 5 ตาราง พร้อม SCD Type 1, 2 และ 3	ครบ
ใช้ EC2, Redshift (PostgreSQL) S3, ALB, Auto Scaling, Lambda, Step Functions, EventBridge, DynamoDB, Systems Manager	ใช้ครบ แต่เปลี่ยนจาก Redshift เป็น RDS เนื่องจากข้อจำกัดด้านการเข้าถึงของ Learner Lab	ครบ
ตั้งค่าให้รองรับ Auto Scaling	Auto Scaling Group พร้อมนโยบาย Target Tracking (CPU 60%)	ครบ
ตั้งค่าให้รองรับ Multi-AZ	EC2 และ ALB กระจาย 2 AZ ส่วน RDS เป็น Single-AZ (DB Subnet Group 2 AZ)	บางส่วน

ตารางที่ 5.2. ผลการดำเนินงานเทียบกับขอบเขตของโครงงาน

เหตุผลของส่วนที่ทำได้บางส่วนหรือปรับเปลี่ยนอธิบายไว้ในหัวข้อ 4.4.4 ของบทที่ 4

5.3. ประโยชน์ของระบบ

- **ผู้บริหารและนักวิเคราะห์ :** เห็นภาพรวมการรับซื้อ การขาย การจัดส่ง และสินค้าคงคลังจากแหล่งข้อมูลเดียว ลดเวลาในการรวบรวมไฟล์จากหลายแผนก
- **ผู้ใช้ทั่วไป :** ถามข้อมูลเป็นภาษาไทยได้โดยไม่ต้องเขียน SQL และตรวจสอบที่มาของตัวเลขได้จากคำสั่ง SQL ที่แสดงประกอบคำตอบ
- **ผู้ดูแลข้อมูล :** อัปโหลดไฟล์ CSV ผ่านหน้าเว็บแล้วระบบทำ ETL ให้อัตโนมัติ ไม่ต้องรันสคริปต์หรือเข้าถึงฐานข้อมูลโดยตรง
- **ความปลอดภัย :** ผู้ใช้แต่ละบทบาทเห็นเฉพาะหน้าที่เกี่ยวข้อง ฐานข้อมูลอยู่ใน Private Subnet และตัวตรวจสอบ SQL อนุญาตเฉพาะการอ่านข้อมูล

5.4. ปัญหาและอุปสรรคที่เกิดขึ้นในการพัฒนา

นอกจากปัญหาทางเทคนิคของโค้ดที่แก้ไขแล้วการพัฒนายังพบอุปสรรคจากสภาพแวดล้อมและกระบวนการทำงานดังนี้

- **ข้อจำกัดของสภาพแวดล้อม :** บัญชี Learner Lab ไม่อนุญาตบางบริการและบางคำสั่ง เช่น Amazon Redshift และการสร้าง IAM Role จึงต้องเปลี่ยนแนวทางออกแบบ และบางคำสั่งถูกปฏิเสธด้วยนโยบายขององค์กร ต้องใช้วิธีทางเลือก เช่น เปลี่ยนเครื่องด้วยตนเอง
- **การเข้าถึงและเวลาในการเปิดระบบ :** รหัส AWS ของ Learner Lab หมดอายุเมื่อสิ้นสุดแต่ละรอบการใช้งาน ต้องขอรหัสชุดใหม่ทุกครั้งที่เปิดแล็บ และทรัพยากรที่ปิดไว้ต้องเปิดตามลำดับ (RDS ก่อน แล้วเปิดเครื่อง EC2) การสร้างและเปิด RDS ใช้เวลา 10-15 นาที ทำให้รอบทดสอบนาน
- **ความแปรผันของ LLM :** ผลลัพธ์ของโมเดลภาษาไม่คงที่ บางครั้งสร้าง SQL อ้างอิง schema ผิด ซึ่ง Guardrail ปฏิเสธถูกต้อง แต่ผู้ใช้เห็นข้อความปฏิเสธ จึงปรับ prompt และบริบท schema ให้สอดคล้องกับค่าตั้งของระบบ
- **ค่าใช้จ่ายที่คิดต่อเนื่อง :** ทรัพยากรบางอย่างคิดค่าใช้จ่ายต่อเนื่องแม้ปิดระบบแล้ว ต้องวางแผนการเปิด-ปิดและลบทรัพยากรให้เหมาะสมกับงบของแล็บ

5.5. ข้อจำกัดของระบบ

- ฐานข้อมูล RDS เป็นแบบ Single-AZ ขนาด db.t3.micro จึงไม่มี failover อัตโนมัติ และไม่เหมาะกับข้อมูลขนาดหลายสิบล้านแถวขึ้นไป
- การเชื่อมต่อผ่าน HTTP (ไม่เข้ารหัส) เครื่อง EC2 อยู่ใน Public Subnet และใช้ LabRole ที่มีสิทธิ์กว้างกว่าที่จำเป็น
- ผู้ช่วย AI พึ่งพาบริการ Groq ผ่านอินเทอร์เน็ต หากบริการหรือคีย์ใช้ไม่ได้จะตอบคำถามเชิงวิเคราะห์ไม่ได้ และจำกัดผลลัพธ์ไม่เกิน 100 แถวต่อคำถาม

- การจัดการผู้ใช้ทำที่ DynamoDB โดยตรง ยังไม่มีหน้าเว็บจัดการผู้ใช้ และสิทธิ์แบ่งตามหน้าเว็บ 3 บทบาทเท่านั้น
- ระบบทำงานเฉพาะเมื่อเปิด Learner Lab และมีค่าใช้จ่ายต่อเนื่องบางส่วนแม้ปิดระบบ หากการใช้งานเพิ่มขึ้น ต้องเพิ่มบริการและค่าใช้จ่ายตามที่วิเคราะห์ในหัวข้อ 5.7

5.6. ประมาณการค่าใช้จ่าย

5.6.1. หลักการและแหล่งข้อมูลที่ใช้ประมาณการ

การประมาณการนี้ใช้ข้อมูล 3 แหล่ง เพื่อให้ตัวเลขอ้างอิงได้และตรวจสอบย้อนกลับได้

1. ข้อมูลการใช้งานและค่าใช้จ่ายจริงของบัญชีจาก AWS Cost Explorer
2. ประวัติการเปิด-ปิดทรัพยากรจริงของโครงงาน ได้แก่ ประวัติกิจกรรมของ Auto Scaling Group เหตุการณ์ของ RDS และเวลาสร้าง ALB และ
3. ราคาทางการ AWS จาก AWS Price List ภูมิภาค US East (N. Virginia) เมื่อวันที่ 2 ตุลาคม 2569 (ไฟล์ราคา EC2 เผยแพร่ 25 กันยายน 2569, RDS 1 ตุลาคม 2569 และ ELB 11 กันยายน 2569)

สมมติฐานที่ใช้ มีดังนี้ ราคาแบบ On-Demand ระบบปฏิบัติการ Linux ภูมิภาค us-east-1 หนึ่งเดือนเท่ากับ 730 ชั่วโมง ไม่นำสิทธิ์ Free Tier และส่วนลดต่าง ๆ มาหัก ไม่รวมภาษีมูลค่าเพิ่ม และแปลงเป็นเงิน บาทที่อัตราสมมติ 1 ดอลลาร์สหรัฐ เท่ากับ 35 บาท เพื่อให้เห็นขนาดโดยประมาณเท่านั้น ค่าใช้จ่ายของ บริการ Groq (LLM) ไม่รวมอยู่ในตัวเลข

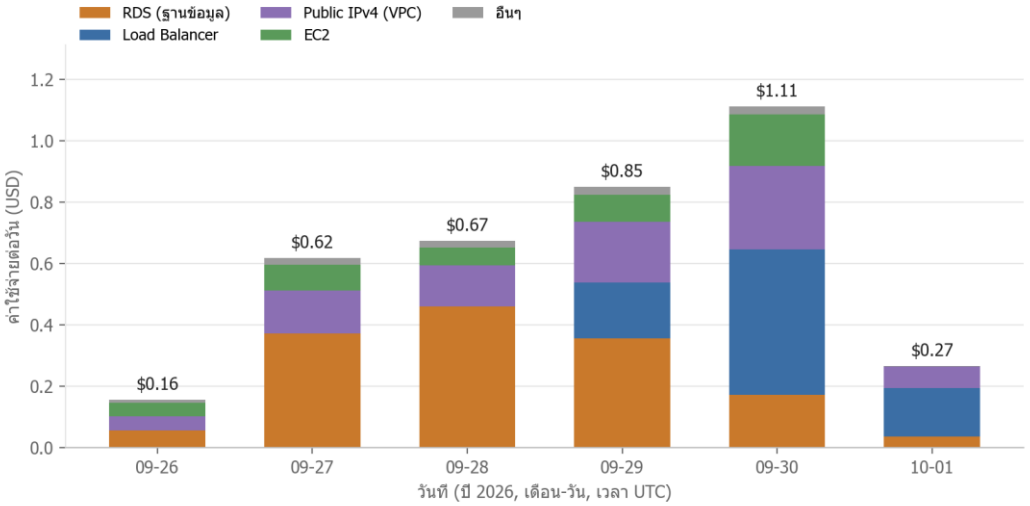
5.6.2. ค่าใช้จ่ายจริงของบัญชีจาก AWS Cost Explorer

ข้อมูลจาก Cost Explorer ช่วงวันที่ 26 กันยายน ถึง 1 ตุลาคม 2569 พบว่ารายการประเภท "Usage" รวม $3.6779 และมีรายการ "Credit" หักลบออกเต็มจำนวน ทำให้ค่าใช้จ่ายสุทธิของบัญชีเป็น $0.00 ซึ่งเป็นไป ตามลักษณะของ AWS Academy Learner Lab ที่ชดเชยค่าใช้จ่ายด้วยเครดิตของแล็บ ดังนั้นตัวเลขที่ใช้ ประเมินคือค่า Usage ก่อนหักเครดิต ตามตารางที่ 5.3 และรูปที่ 5.1

บริการ	ค่าใช้จ่าย Usage (USD)	สัดส่วน
Amazon Relational Database Service (RDS)	1.4573	39.6%
Amazon Virtual Private Cloud (Public IPv4)	0.8606	23.4%
Elastic Load Balancing	0.8108	22.0%
EC2 – Compute	0.4420	12.0%
EC2 – Other (EBS)	0.0954	2.6%
Amazon CloudWatch	0.0096	0.3%
Amazon S3	0.0018	0.0%

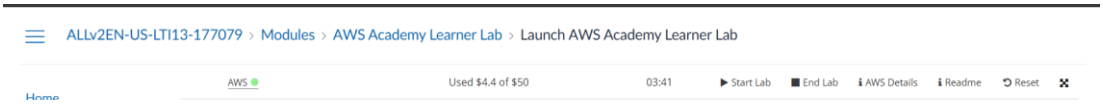
AWS KMS	0.0003	0.0%
Amazon DynamoDB	0.0001	0.0%
CloudWatch Events (EventBridge)	0.0001	0.0%
รวม	3.6779	100%

ตารางที่ 5.3. ค่าใช้จ่ายจริงของบัญชีจาก Cost Explorer แยกตามบริการ (26 ก.ย. – 1 ต.ค. 2569)



รูปที่ 5.1. ค่าใช้จ่ายจริงรายวันของบัญชีแยกตามบริการ (ข้อมูล Cost Explorer ก่อนหักเครดิต)

ข้อสังเกต ฐานข้อมูลคิดเป็นสัดส่วนสูงสุดช่วงก่อนวันที่ 29 กันยายน เนื่องจากเป็นฐานข้อมูลรุ่นทดลองของทีมงานที่ไม่ได้อยู่ใน stack agrichain (ชนิด db.t4g.micro ใช้งาน 59.35 ชั่วโมง ค่าใช้จ่าย $0.95) และ หลังติดตั้ง stack วันที่ 29 กันยายน ค่า Load Balancer และ Public IPv4 เพิ่มขึ้นชัดเจน และสุดท้าย ในวันที่ 1 ตุลาคม ซึ่งปิดเครื่อง EC2 และ RDS แล้ว ค่าใช้จ่ายยังคงอยู่ราว $0.27 ต่อวัน จาก ALB และ Public IPv4 ที่ยังคิดต่อเนื่อง



รูปที่ 5.2. งบประมาณของ Learner Lab : ใช้ไปแล้ว $4.4 จากงบ $50

ตัวนับของ Learner Lab แสดง "Used $4.4 of $50" คือใช้ไปแล้วประมาณ 8.8% ของงบ ตัวเลขนี้สูงกว่า $3.68 ของ Cost Explorer เล็กน้อย ซึ่งเป็นเพราะข้อมูล Cost Explorer ที่ดึงมาสิ้นสุดวันที่ 1 ตุลาคม ขณะที่ตัวนับของแล็บอัปเดตล่าสุดกว่า

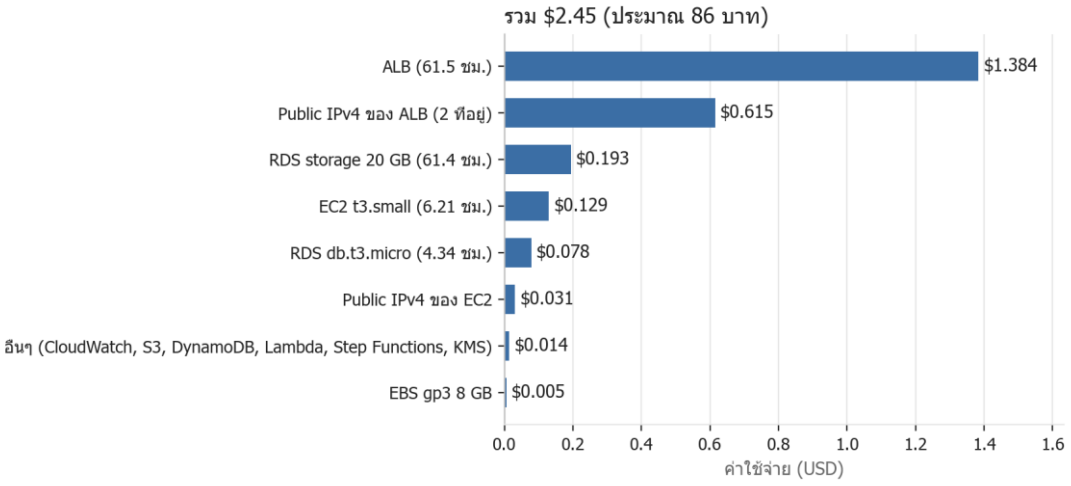
5.6.3. ค่าใช้จ่ายของโครงงาน AgriChain ถึงปัจจุบัน

เพื่อแยกเฉพาะส่วนของ stack agrichain ออกจากทรัพยากรอื่นในบัญชี ผู้จัดทำคำนวณจากจำนวนชั่วโมงที่ทรัพยากรทำงานจริงคูณราคาต่อหน่วย โดยชั่วโมงของเครื่อง EC2 ได้จากประวัติกิจกรรมของ Auto Scaling Group (เครื่องที่ถูกสร้างทั้งหมด 16 เครื่อง ระยะเวลาทำงานรวม 6.21 ชั่วโมง) ชั่วโมงของ

RDS ได้จากเหตุการณ์ created, started และ stopped (รวม 4.34 ชั่วโมง ซึ่งใกล้เคียงกับ 3.95 ชั่วโมงในรายงาน Cost Explorer ของ db.t3.micro) และชั่วโมงของ ALB นับจากเวลาสร้าง (61.5 ชั่วโมง ณ วันที่ 2 ตุลาคม 2569 เวลา 05:34 UTC) พื้นที่เก็บข้อมูลของ RDS ขนาด 20 GB คิดค่าตลอดเวลาที่ฐานข้อมูลมีอยู่ แม้หยุดการทำงาน ผลการคำนวณแสดงในตารางที่ 5.4 และรูปที่ 5.3

รายการ	ค่าใช้จ่าย (USD)
EC2 t3.small (6.21 ชม.)	0.1292
EBS gp3 8 GB	0.0054
RDS db.t3.micro (4.34 ชม.)	0.0781
RDS storage 20 GB (61.4 ชม.)	0.1935
ALB (61.5 ชม.)	1.3838
Public IPv4 ของ ALB (2 ที่อยู่)	0.6150
Public IPv4 ของ EC2	0.0311
อื่นๆ (CloudWatch, S3, DynamoDB, Lambda, Step Functions, KMS)	0.0143
รวมโดยประมาณ	2.4503
คิดเป็นเงินบาท (อัตราสมมติ 35 บาท/ดอลลาร์)	86 บาท

ตารางที่ 5.4 ค่าใช้จ่ายโดยประมาณของ stack agrichain ตั้งแต่ติดตั้งจนถึง 2 ต.ค. 2569



รูปที่ 5.3. องค์ประกอบค่าใช้จ่ายของโครงงาน AgriChain ถึงวันที่ 2 ตุลาคม 2569

ส่วนที่ใหญ่ที่สุด เป็น Application Load Balancer ($1.38) และค่า Public IPv4 ของ ALB ($0.61) ซึ่งคิดตามเวลาที่ ALB มีอยู่ ไม่ขึ้นกับว่ามีผู้ใช้หรือไม่ ในขณะที่เครื่อง EC2 และ RDS ถูกเปิดเพียงไม่กี่ชั่วโมงเฉพาะช่วงพัฒนาและทดสอบ ค่าใช้จ่ายรวมจึงต่ำเมื่อเทียบกับการเปิดระบบตลอดเวลา

5.6.4. ประมาณการรายเดือนกรณีเปิดระบบตลอด 24 ชั่วโมง

หากเปิดระบบตลอดเดือน (730 ชั่วโมง) ค่าใช้จ่ายขึ้นกับจำนวนเครื่องและชนิดของทรัพยากร ตารางที่ 5.5 แสดงการคำนวณทีละรายการสำหรับ 2 กรณี กรณีแรกคือการติดตั้งจริงที่มีเครื่อง EC2 เครื่องเดียว และกรณีที่สองคือตามแผนภาพที่ออกแบบให้มี 2 เครื่องใน 2 Availability Zone

รายการ	วิธีคำนวณ	กรณี 1 เครื่อง	กรณี 2 เครื่อง
EC2 t3.small	730 ชม. × $0.0208 × เครื่อง	$15.18	$30.37
EBS gp3 8 GB ต่อเครื่อง	8 GB × $0.08 × เครื่อง	$0.64	$1.28
RDS db.t3.micro + ที่เก็บ 20 GB	730 × $0.018 + 20 × $0.115	$15.44	$15.44
Application Load Balancer	730 × $0.0225 + LCU	$16.45	$16.45
Public IPv4	730 × $0.005 × (2 ของ ALB + 1 ต่อเครื่อง EC2)	$10.95	$14.60
อื่นๆ (CloudWatch, DynamoDB, S3 และอื่น)	ตามการใช้งานจริง	$0.28	$0.28
รวมต่อเดือน (USD)		$58.94	$78.41
รวมต่อเดือน (บาท)	อัตราสมมติ 35 บาท/ดอลลาร์	2,063	2,744

ตารางที่ 5.5 ประมาณการรายเดือนเมื่อเปิดระบบตลอด 24 ชั่วโมง

Lambda, Step Functions, EventBridge, DynamoDB, และ Gateway VPC Endpoint มีค่าใช้จ่ายต่ำมากจนไม่ทำให้ตัวเลขรวมเปลี่ยนอย่างมีนัยสำคัญ ทั้งหมดอยู่ในโควตาฟรีรายเดือนของ AWS นอกจากนี้ S3 เก็บข้อมูลรวมประมาณ 0.28 GB ค่าใช้จ่ายไม่ถึง $0.01 ต่อเดือน

5.6.5. แนวทางลดค่าใช้จ่าย

แนวทาง	หลักการ	ผลต่อค่าใช้จ่าย
เปิดระบบเฉพาะเวลาใช้งาน	หยุด EC2 (ตั้ง Auto Scaling เป็น 0) หยุด RDS เมื่อไม่ใช้	ชั่วโมงเครื่องและฐานข้อมูลลดจาก 730 เหลือ 40 ชั่วโมง/เดือน
ลบหรือหยุดสร้าง ALB เมื่อไม่ใช้งานนาน	ALB และ Public IPv4 คิดตามเวลาที่มีอยู่	ประหยัดประมาณ $23.73 /เดือน
ใช้ฐานข้อมูลรุ่น Graviton (db.t4g.micro)	ราคาต่อชั่วโมงต่ำกว่ารุ่น x86 ที่ขนาดใกล้เคียง	$0.016 เทียบกับ $0.018 ต่อชั่วโมง ลดประมาณ 11%

ใช้เครื่อง EC2 ให้พอดีกับภาระงาน	เริ่มจาก 1 เครื่อง และ Auto Scaling เพิ่มเมื่อ CPU เกิน 60%	ลดค่าเครื่องและ IPv4 ประมาณ $19.47 ต่อเครื่องต่อเดือน เมื่อเทียบกับการคงไว้ 2 เครื่อง
ใช้ Gateway Endpoint แทน NAT Gateway	เข้าถึง S3 และ DynamoDB จาก Private Subnet โดยไม่ออกอินเทอร์เน็ต	ไม่มีค่า Endpoint ประเภท Gateway และหลีกเลี่ยง NAT Gateway ประมาณ $33.75 ต่อเดือน (ทำแล้วในระบบนี้)
ใช้ส่วนลดแบบผูกพันเมื่อใช้จริงระยะยาว	Savings Plans หรือ Reserved Instances	ลดตามเงื่อนไขที่ AWS กำหนด ต้องใช้ตารางราคาล่าสุด

ตารางที่ 5.6 แนวทางลดค่าใช้จ่ายและผลโดยประมาณ

5.7. ข้อเสนอแนะและแนวทางพัฒนาต่อ

- **ด้านการใช้งาน :** เพิ่มหน้าเว็บแสดงสถานะและข้อผิดพลาดของ ETL ต่อไฟล์ (ดึงจาก Step Functions) เพื่อให้ผู้ดูแลข้อมูลตรวจสอบได้โดยไม่ต้องเข้า AWS Console
- **ด้านข้อมูล :** ปรับ ETL ให้ป้องกันข้อมูลซ้ำ (idempotent) และปิดเวอร์ชันเก่าของ SCD Type 2 รวมทั้งเพิ่มการตรวจสอบรูปแบบไฟล์และรายงานแถวที่ผิดพลาดก่อนโหลด
- **ด้านสถาปัตยกรรม :** ทำ Load test เพื่อยืนยันการ scale-out ของ Auto Scaling และกำหนดค่า CPU เป้าหมาย รวมทั้งแยกชนิดข้อผิดพลาดที่ Step Functions ควร Retry ออกจากข้อผิดพลาดของข้อมูล
- **ด้าน AI :** ทดสอบความถูกต้องของ Text-to-SQL ด้วยชุดคำถามมาตรฐานเทียบกับคำตอบที่ตรวจแล้ว และเพิ่มการจำกัดสิทธิ์ข้อมูลตามบทบาทในระดับตารางหรือแถว
- **ด้านค่าใช้จ่าย :** ตั้งแนวปฏิบัติเปิด-ปิดระบบและงบประมาณ (AWS Budgets พร้อมการแจ้งเตือน) และพิจารณาลบ ALB เมื่อไม่ใช้งานนาน

5.8. สรุป

โครงงาน AgriChain พัฒนาและติดตั้งแพลตฟอร์มคลังข้อมูลสำหรับห่วงโซ่อุปทานสินค้าเกษตรบน AWS ได้ตามวัตถุประสงค์หลัก ระบบรับไฟล์ CSV นำเข้าผ่านเว็บ ทำ ETL อัตโนมัติด้วย S3, Step Functions และ Lambda จัดเก็บในฐานข้อมูล Star Schema บน RDS ให้บริการผ่าน Application Load Balancer และ Auto Scaling Group ใน 2 Availability Zone มีผู้ช่วย AI ตอบคำถามภาษาไทยพร้อมกราฟ และแบ่งสิทธิ์ผู้ใช้ตาม Use Case ค่าใช้จ่ายจริงของโครงงานตลอดช่วงพัฒนาและทดสอบอยู่ที่ประมาณ $2.45 (ประมาณ 86 บาท) ซึ่งต่ำเพราะเปิดเครื่องประมวลผลเฉพาะช่วงทดสอบ (ทั้งบัญชีใช้เครดิตไป $4.4 จากงบ $50 ตามตัวนับของ Learner Lab) ทั้งนี้หากเปิดระบบตลอดเวลาจะมีค่าใช้จ่ายประมาณ $58.94 ถึง $78.41 ต่อเดือน

บรรณานุกรม

[1] AWS Academy. (n.d.). AWS Academy Learner Lab [Course]. Amazon Web Services Training and Certification.

[2] Amazon Web Services. (n.d.). Amazon CloudWatch pricing. https://aws.amazon.com/cloudwatch/pricing/

[3] Amazon Web Services. (n.d.). Amazon DynamoDB pricing for on-demand capacity. https://aws.amazon.com/dynamodb/pricing/on-demand/

[4] Amazon Web Services. (n.d.). Amazon EC2 on-demand pricing. https://aws.amazon.com/ec2/pricing/on-demand/

[5] Amazon Web Services. (n.d.). Amazon RDS for PostgreSQL pricing. https://aws.amazon.com/rds/postgresql/pricing/

[6] Amazon Web Services. (n.d.). Amazon Redshift pricing. https://aws.amazon.com/redshift/pricing/

[7] Amazon Web Services. (n.d.). Amazon S3 event notifications with Amazon EventBridge. https://docs.aws.amazon.com/AmazonS3/latest/userguide/EventBridge.html

[8] Amazon Web Services. (n.d.). Amazon S3 pricing. https://aws.amazon.com/s3/pricing/

[9] Amazon Web Services. (n.d.). Amazon VPC pricing. https://aws.amazon.com/vpc/pricing/

[10] Amazon Web Services. (n.d.). Application Load Balancers. https://docs.aws.amazon.com/elasticloadbalancing/latest/application/introduction.html

[11] Amazon Web Services. (n.d.). AWS Cost Explorer. https://aws.amazon.com/aws-cost-management/aws-cost-explorer/

[12] Amazon Web Services. (n.d.). AWS Lambda pricing. https://aws.amazon.com/lambda/pricing/

[13] Amazon Web Services. (n.d.). AWS Price List bulk files (us-east-1). https://pricing.us-east-1.amazonaws.com/offers/v1.0/aws/index.json

[14] Amazon Web Services. (n.d.). AWS Pricing Calculator. https://calculator.aws/

ภาคผนวก

ภาคผนวก ก โครงสร้างไฟล์ของโครงการ

โครงสร้างหลักของที่เก็บโค้ด (repository) ของโครงการ แสดงเฉพาะไฟล์และโฟลเดอร์สำคัญ

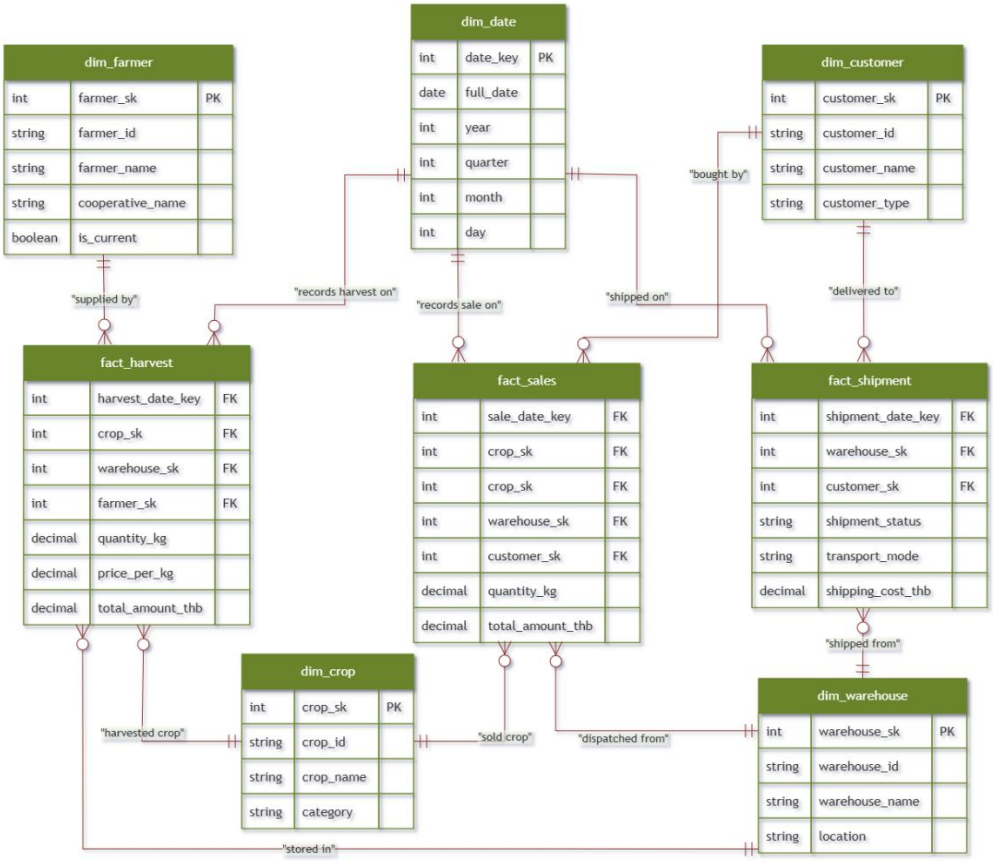
```
CLOUD/
├──── server.js    เว็บเซิร์ฟเวอร์ Express: ล็อกอิน, สิทธิ์ตามบทบาท, อัปโหลด S3, พร็อกซีไป AI
├──── package.json
├──── frontend/              หน้าเว็บ
│     ├──── index.html, login.js หน้าเข้าสู่ระบบ
│     ├──── dashboard.html     แดชบอร์ด
│     ├──── inventory.html     หน้าคลังสินค้า (อัปโหลด CSV)
│     ├──── agent.html         ผู้ช่วย AI
│     └──── auth-guard.js      ตรวจ 401 และซ่อนเมนูตามบทบาท
├──── app/                   AI Agent (FastAPI)
│     ├──── main.py, api/routes.py
│     ├──── agents/            สร้างแผนคำค้น (Text-to-SQL)
│     ├──── providers/         เชื่อมต่อ LLM (Groq/Qwen)
│     ├──── guardrails/sql_guard.py   ตรวจ SQL (อ่านอย่างเดียว, จำกัดแถว)
│     ├──── db/                การเชื่อมต่อและ schema catalog
│     ├──── services/          เวิร์กโฟลว์ตอบคำถาม
│     ├──── visualization/     สร้างกราฟ
│     └──── prompts/           system prompt และ prompt Text-to-SQL
├──── lambda/etl/
│     ├──── lambda_function.py     ETL + Custom Resource (สร้าง schema, ผู้ใช้ตั้งต้น)
│     ├──── schema.sql             โครงสร้างตารางคลังข้อมูล
│     └──── requirements.txt
├──── dataset/             ไฟล์ CSV ทดสอบ 8 ไฟล์ (master/, operational/)
```

ภาคผนวก ข โครงสร้างฐานข้อมูลคลังข้อมูล (Star Schema)

คลังข้อมูลอยู่ใน schema data_warehouse ประกอบด้วยตารางมิติ 5 ตารางและตารางข้อเท็จจริง 4 ตารางซึ่งเป็นไปตาม ER Diagram ตามรูปที่.1 ตารางข้อเท็จจริงเชื่อมกับตารางมิติด้วย surrogate key ตัวอย่างคำสั่ง SQL ของตารางข้อเท็จจริง (จากไฟล์ lambda/etl/schema.sql) แสดงดังนี้

```
-- FACT TABLES
-- fact_harvest  (grain: 1 row = 1 รายการรับซื้อ)
CREATE TABLE IF NOT EXISTS fact_harvest (
    harvest_id        VARCHAR(20) PRIMARY KEY,
    harvest_date_key   INTEGER NOT NULL REFERENCES dim_date(date_key),
    farmer_sk         INTEGER NOT NULL REFERENCES dim_farmer(farmer_sk),
    crop_sk           INTEGER NOT NULL REFERENCES dim_crop(crop_sk),
    warehouse_sk       INTEGER NOT NULL REFERENCES dim_warehouse(warehouse_sk),
    quantity_kg       NUMERIC(14,2) NOT NULL,
    price_per_kg       NUMERIC(12,2) NOT NULL,
    total_amount_thb   NUMERIC(16,2) NOT NULL,
    quality_grade      VARCHAR(5)
);
-- fact_sales  (grain: 1 row = 1 รายการขาย)
CREATE TABLE IF NOT EXISTS fact_sales (
    sales_id          VARCHAR(20) PRIMARY KEY,
    sale_date_key     INTEGER NOT NULL REFERENCES dim_date(date_key),
    customer_sk       INTEGER NOT NULL REFERENCES dim_customer(customer_sk),
    crop_sk           INTEGER NOT NULL REFERENCES dim_crop(crop_sk),
    warehouse_sk      INTEGER NOT NULL REFERENCES dim_warehouse(warehouse_sk),
    quantity_kg       NUMERIC(14,2) NOT NULL,
    unit_price_thb    NUMERIC(12,2) NOT NULL,
    total_amount_thb  NUMERIC(16,2) NOT NULL,
    discount_pct      NUMERIC(5,2),
    sale_channel      VARCHAR(50));
-- fact_shipment  (grain: 1 row = 1 รอบการจัดส่ง)
CREATE TABLE IF NOT EXISTS fact_shipment (
    shipment_id        VARCHAR(20) PRIMARY KEY,
    shipment_date_key   INTEGER NOT NULL REFERENCES dim_date(date_key),
    delivery_date_key   INTEGER NOT NULL REFERENCES dim_date(date_key),
    customer_sk        INTEGER NOT NULL REFERENCES dim_customer(customer_sk),
    crop_sk            INTEGER NOT NULL REFERENCES dim_crop(crop_sk),
```

```
    warehouse_sk        INTEGER NOT NULL REFERENCES dim_warehouse(warehouse_sk),
    total_weight_kg     NUMERIC(14,2) NOT NULL,
    shipping_cost_thb   NUMERIC(14,2),
    status              VARCHAR(50),
    transport_mode      VARCHAR(50) );
-- fact_inventory  (grain: 1 row = 1 บันทึกยอดคงเหลือต่อคลัง)
CREATE TABLE IF NOT EXISTS fact_inventory (
    inventory_id        VARCHAR(20) PRIMARY KEY,
    snapshot_date_key   INTEGER NOT NULL REFERENCES dim_date(date_key),
    warehouse_sk        INTEGER NOT NULL REFERENCES dim_warehouse(warehouse_sk),
    crop_sk             INTEGER NOT NULL REFERENCES dim_crop(crop_sk),
    beginning_stock_kg NUMERIC(14,2) NOT NULL,
    received_kg         NUMERIC(14,2) NOT NULL,
    sold_kg             NUMERIC(14,2) NOT NULL,
    ending_stock_kg     NUMERIC(14,2) NOT NULL,
    unit_cost_thb       NUMERIC(12,2) );
```



รูปที่ข.1. ER Diagram คลังข้อมูล

ภาคผนวก ค ตัวอย่างโค้ด ETL บน AWS Lambda

Lambda ชื่อ agrichain-etl เลือกฟังก์ชัน ETL จากคำในชื่อไฟล์ ทำความสะอาดข้อมูล แปลงรหัสธุรกิจเป็น surrogate key และโหลดเข้า RDS

```
def get_surrogate_key_mapping(engine, table_name, sk_col, bk_col):
    query = f"SELECT {sk_col}, {bk_col} FROM {DB_SCHEMA}.{table_name}"
    if table_name in ("dim_farmer", "dim_customer"):
        query += " WHERE is_current = true"
    df_lookup = pd.read_sql(query, con=engine)
    return dict(zip(df_lookup[bk_col], df_lookup[sk_col]))
def load(df, engine, table):
    cols = pd.read_sql( text("SELECT column_name FROM information_schema.columns "
            "WHERE table_schema = :s AND table_name = :t"),
        con=engine, params={"s": DB_SCHEMA, "t": table},)["column_name"].tolist()
    dropped = [c for c in df.columns if c not in cols]
    if dropped:
        print(f"{table}: ignoring source-only columns {dropped}")
    df = df[[c for c in df.columns if c in cols]]
    df.to_sql(name=table, con=engine, schema=DB_SCHEMA, if_exists="append", index=False,
      method="multi",chunksize=500,)
    return table
def date_key(series):
    return series.astype(str).str.slice(0, 10).str.replace("-", "", regex=False).astype(int)
def map_keys(df, engine, specs):
    for table, sk_col, bk_col in specs:
        df[sk_col] = df[bk_col].map(get_surrogate_key_mapping(engine, table, sk_col, bk_col))
        missing = df[sk_col].isna()
        if missing.any():
            bad = sorted(df.loc[missing, bk_col].astype(str).unique())[:10]
            raise ValueError(
                f"{missing.sum()} row(s) reference {bk_col} not found in {table} "
                f"(e.g. {bad}). Upload the dimension files first.")
        df[sk_col] = df[sk_col].astype(int)
    return df
def etl_shipment(df, engine):
    df["shipment_date_key"] = date_key(df["shipment_date"])
    df["delivery_date"] = df["delivery_date"].fillna("9999-12-31")
```

```
    df["delivery_date_key"] = date_key(df["delivery_date"])
    df = map_keys(df, engine, [
        ("dim_customer", "customer_sk", "customer_id"),
        ("dim_crop", "crop_sk", "crop_id"),
        ("dim_warehouse", "warehouse_sk", "warehouse_id"),])
    df = df.drop(columns=[
        "customer_id", "crop_id", "warehouse_id", "shipment_date",
        "delivery_date", "driver_name", "vehicle_plate",])
    return load(df, engine, "fact_shipment")
def etl_inventory(df, engine):
    df["snapshot_date_key"] = date_key(df["snapshot_date"])
    df["received_kg"] = df["received_kg"].fillna(0.00)
    df["sold_kg"] = df["sold_kg"].fillna(0.00)
    df = map_keys(df, engine, [
        ("dim_crop", "crop_sk", "crop_id"),
        ("dim_warehouse", "warehouse_sk", "warehouse_id"), ])
    df = df.drop(columns=["crop_id", "warehouse_id", "snapshot_date", "stock_status"])
    return load(df, engine, "fact_inventory")
ROUTES = [ ("Crop", etl_crop), ("Farmer", etl_farmer),  ("Warehouse", etl_warehouse),  ("Customer", etl_customer),
("Harvest", etl_harvest),  ("Sales", etl_sales), ("Shipment", etl_shipment), ("Inventory", etl_inventory),]
def run_etl(event):
    bucket, key = extract_s3_object(event)
    print(f"ETL start: s3://{bucket}/{key}")
    if not key.lower().endswith(".csv"):
        print(f"Skipping non-CSV object: {key}")
        return {"statusCode": 200, "body": f"Skipped non-CSV object {key}"}
    filename = key.rsplit("/", 1)[-1]
    handler = next((fn for word, fn in ROUTES if word in filename), None)
    if handler is None:
        raise ValueError(f"File pattern not recognized: {key}")
    obj = s3_client.get_object(Bucket=bucket, Key=key)
    df = pd.read_csv(obj["Body"], encoding="utf-8-sig")  # source CSVs carry a BOM
    target_table = handler(df, get_db_engine())
    msg = f"ETL completed for {key}: inserted {len(df)} rows into {DB_SCHEMA}.{target_table}"
    print(msg)
    return {"statusCode": 200, "body": msg, "table": target_table, "rows": int(len(df))}
```

ภาคผนวก ง คู่มือการติดตั้งและการเปิด-ปิดระบบ

การติดตั้งครั้งแรก

ต้องมี Python 3 และ boto3 และตั้งรหัสเข้าถึงจากหน้า AWS Details ของ Learner Lab

```
python -m pip install boto3
export AWS_ACCESS_KEY_ID=...        # จากหน้า AWS Details ของ Learner Lab
export AWS_SECRET_ACCESS_KEY=...
export AWS_SESSION_TOKEN=...
export AWS_DEFAULT_REGION=us-east-1
python infra/deploy.py --groq-key <GROQ_API_KEY> \
    --admin-username exec01 --admin-name exec01 --admin-password "<รหัสผ่าน>"
```

การปิดระบบชั่วคราว

```
aws autoscaling update-auto-scaling-group --auto-scaling-group-name agrichain-asg --min-size 0 --desired-capacity 0
aws rds stop-db-instance --db-instance-identifier agrichain-dwh-db
```

การเปิดระบบกลับ

```
aws rds start-db-instance --db-instance-identifier agrichain-dwh-db        # รอสถานะ available
aws autoscaling update-auto-scaling-group --auto-scaling-group-name agrichain-asg --min-size 1 --desired-capacity 1
```

การปล่อยโค้ดเวอร์ชันใหม่

รันคำสั่งติดตั้งเดิมซ้ำ (อัปเดต stack เดิม) จากนั้นต้องเปลี่ยนเครื่อง EC2 ให้ใช้โค้ดใหม่ โดยเพิ่ม จำนวนเครื่องแล้วยกเลิกเครื่องเก่าทีละตัว (คำสั่ง Instance Refresh ถูกจำกัดสิทธิ์ใน Learner Lab)

ภาคผนวก จ พจนานุกรมข้อมูลชุดทดสอบ

ชุดข้อมูลทดสอบเป็นข้อมูลจำลองที่สร้างจากสคริปต์ generate_dataset.py (กำหนด seed คงที่) ครอบคลุมช่วงปี 2021–2025 ประกอบด้วยไฟล์ CSV 8 ไฟล์ รวม 170,504 แถว ตามตารางที่

ไฟล์	ตารางปลายทาง	แถว	คอลัมน์ในไฟล์ต้นทาง
Crop.csv	dim_crop	15	crop_id, crop_name, category, unit, standard_price_per_unit, season_months, shelf_life_days, is_active*
Farmer.csv	dim_farmer	300	farmer_id, farmer_name, national_id*, province, region, district*, cooperative,

			farm_size_rai, primary_crop, registration_date*, status*, phone*
Warehouse.csv	dim_warehouse	15	warehouse_id, warehouse_name, province, region, address*, capacity_ton, current_utilization_pct*, manager_name, contact_phone*, is_active*
Customer.csv	dim_customer	200	customer_id, customer_name, customer_type, province, region, contact_person*, phone*, email*, credit_limit_thb, is_active*
Harvest.csv	fact_harvest	50,000	harvest_id, farmer_id, crop_id, warehouse_id, harvest_date, quantity_kg, price_per_kg, total_amount_thb, quality_grade, inspector_note*
Sales.csv	fact_sales	30,000	sales_id, order_id*, customer_id, crop_id, warehouse_id, sale_date, quantity_kg, unit_price_thb, total_amount_thb, discount_pct, payment_status*, sale_channel
Shipment.csv	fact_shipment	25,000	shipment_id, warehouse_id, customer_id, crop_id, shipment_date, delivery_date, status, total_weight_kg, shipping_cost_thb, transport_mode, driver_name*, vehicle_plate*
Inventory.csv	fact_inventory	64,974	inventory_id, warehouse_id, crop_id, snapshot_date, beginning_stock_kg, received_kg, sold_kg, ending_stock_kg, unit_cost_thb, stock_status*

ตารางที่ จ.1. ไฟล์ข้อมูลทดสอบและคอลัมน์ (* คือคอลัมน์ที่ไม่โหลดเข้าคลังข้อมูล)